

claude-windows / 50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a\subagents\workflows\wf_ddb0b3fd-b40\agent-af21c0f5d4590409e.jsonl`
- SHA-256: `419b720f589936deedf76766ed14d6d0835d9d1e18dca9df7a8fb1d8d6d4a753`
- Source modified: `2026-07-07T04:14:53+00:00`
- Imported at: `2026-07-08T16:00:26+00:00`
- project: `wf_ddb0b3fd-b40`
- session_id: `50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a`
```

Transcript

1. user (2026-07-07T04:01:30.639Z)

You are reviewing an UNCOMMITTED working diff implementing issue #506 (GPU-resident WASB inference) in the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer. The full diff is at C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a/scratchpad/506_diff.patch (read it). The new test file tests/test_wasb_nvdec_resident.py is untracked (not in the diff) ? read it directly from the repo.

Ground truth you must compare against:

- The VALIDATED reference implementation: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/scratch/gpu_resident/exp_t2_full.py (the GpuFrontend class + its grid build ARE what was validated on a real L4: F1 0.574 ? baseline 0.582 on the hard clip). The production port must be semantically faithful to it.
- WASB-SBDT interfaces (not in this repo; trust these facts):
TracknetV2Detector.run_tensor(imgs, affine_mats) moves imgs .to('cuda') itself and calls postprocessor.run(preds, affine_mats) which does affine_mats[scale].cpu().numpy() then indexes [i] per batch item; ImageDataset (eval) returns (imgs_t, hms_t, trans_outputs_inv, xys, visis, img_paths) where trans_outputs_inv is {scale: np.float64 [2,3]}; per-frame preprocessing is PIL RGB -> cv2.warpAffine(trans_input, (512,288), INTER_LINEAR) -> ToTensor -> Normalize(ImageNet); get_transform uses c=(w/2,h/2), s=max(h,w); OnlineTracker uses np.Inf; WASB detector.py calls torch.load(model_path) with no weights_only arg.
- The CPU decode path (run(), run_video_streaming) must be BEHAVIOR-UNCHANGED on the WSL torch-1.11 + numpy<1.24 env (byte-parity guarantee), except that torch>=2.6 envs get the weights_only + np.Inf shims.

Report ONLY defects you can ground in the actual code you read ? cite file:line. No style nits, no speculative maybes without a concrete failure scenario. Return findings as structured output.

DIMENSION: test validity + deploy env changes. Scrutinize tests/test_wasb_nvdec_resident.py (is _wasb_affine_matrices a FAITHFUL replica of WASB's get_affine_transform for rot=0 ? check get_3rd_point/dst_dir math; do the CLI tests actually exercise main()'s branch before heavy imports; is the GPU parity test's staging/import approach sound ? hydra config_path resolution relative to the staged file, module-name collision with the already-imported backend wasb_infer, sys.path cleanup), tests/test_config_wasb.py + tests/test_native_wasb_runner.py additions (do they assert what their docstrings claim?), tests/test_compute_backend.py Dockerfile guard update, deploy/cloudrun/Dockerfile (multi-stage ARG scoping rules ? is NVIDIA_DRIVER_VERSION actually available in both stages as written? COPY --from correctness, symlink targets, does the wasb env build still satisfy test_dockerfile_uses_micromamba_not_anaconda_tos?), and deploy/digitalocean/gpu_setup.sh (set -euo pipefail interactions with the new step: nvidia-smi missing, curl fallback chain, ln -sf under sudo, does the || true placement change error behavior?).

2. user (2026-07-07T04:01:37.016Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-875 of 1092 total (26506 tokens, cap

25000). Call Read with offset=876 limit=875 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-reminder>

```

1      diff --git a/backend/config/models.py b/backend/config/models.py
2      index cd6f2fc..a7c997c 100644
3      --- a/backend/config/models.py
4      +++ b/backend/config/models.py
5      @@ -119,6 +119,28 @@ class WasbIndexConfig(BaseModel):
6          # Depth of the bounded producer-thread queue that overlaps CPU-side batch assembly
7          # (decode + PIL + ToTensor) with GPU inference in streaming mode. None/0 = off.
8          prefetch_batches: Optional[int] = None
9      +    # GPU-RESIDENT DECODE (#506) ? DEFAULT OFF. "nvdec_resident" moves
decode+resize+normalize
10     +    # onto the GPU with no host round-trip: PyNvVideoCodec NVDEC decode ? affine
`grid_sample`
11     +    # resize replicating WASB's cv2.warpAffine EXACTLY (the #506 lesson: a naive
F.interpolate
12     +    # silently over-detects ? F1 0.58?0.07 ? and only shows on hard clips) ? GPU
ImageNet
13     +    # normalize ? HRNet. Validated on a real L4 (GX010128, SERVED preset, 2026-07-06):
~1.7x
14     +    # fps (35.8?60.2), GPU util 47?79% / CPU 62?35%, F1 0.574 ? baseline 0.582. Native
(Linux)
15     +    # runner only ? needs the torch-2.x + PyNvVideoCodec WASB env
(deploy/cloudrun/Dockerfile)
16     +    # with libnvcuvid/libnvidia-encode present, and device=cuda; the WSL runner ignores
it.
17     +    # Flip gate: the full 15-clip golden set at the SERVED preset (GX010128 must pass).
18     +    decode_backend: Literal["cpu", "nvdec_resident"] = "cpu"
19     +
20     +    @model_validator(mode="after")
21     +    def _nvdec_resident_requires_cuda(self) -> "WasbIndexConfig":
22     +        # NVDEC decode is CUDA-only; reject the contradictory combo at config load
instead of
23     +        # failing deep inside a paid GPU run. "auto" is allowed here ? the native
runner's
24     +        # healthcheck hard-fails if auto resolves to cpu with this backend on.
25     +        if self.decode_backend == "nvdec_resident" and self.device not in ("cuda",
"auto"):
26     +            raise ValueError(
27     +                f"indexing.wasb.decode_backend='nvdec_resident' requires device 'cuda'
or "
28     +                f"'auto' (got {self.device!r}) ? NVDEC decode runs on the GPU."
29     +            )
30     +        return self
31
32     @field_validator("batch_size")
33     @classmethod
34     diff --git a/backend/pipeline/detectors/native_wasb_runner.py
b/backend/pipeline/detectors/native_wasb_runner.py
35     index 54cec69..230d889 100644
36     --- a/backend/pipeline/detectors/native_wasb_runner.py
37     +++ b/backend/pipeline/detectors/native_wasb_runner.py
38     @@ -60,6 +60,10 @@ _CUDA_PROBE_CODE = (
39         "import torch; print('torch', torch.__version__, 'cuda',
torch.cuda.is_available())"
40     )
41
42     + # Decode backends wasb_infer.py understands (#506). Mirrored by its --decode-backend
choices;
43     + # healthcheck rejects anything else up front so a typo can't reach argparse as rc=2
mid-run.
44     + _VALID_DECODE_BACKENDS = ("cpu", "nvdec_resident")
45     +

```

```

46
47     @dataclass
48     class NativeWasbConfig:
49         @@ -90,6 +94,10 @@ class NativeWasbConfig:
50             # wasb_infer.py's parity defaults (batch 8, no prefetch). The cloud-serving preset
51             batch_size: int = 0
52             prefetch_batches: int = 0
53             + # GPU-resident NVDEC decode (#506, indexing.wasb.decode_backend /
54             WASB_DECODE_BACKEND).
55             + # "cpu" (default) = omit the flag ? wasb_infer.py's historical cv2 decode paths,
56             + # byte-identical. "nvdec_resident" = decode+resize+normalize on-GPU (torch-2.x env
57             + # only).
58             + decode_backend: str = "cpu"
59
60             @classmethod
61             def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "NativeWasbConfig":
62                 @@ -115,6 +123,7 @@ class NativeWasbConfig:
63                     stream_video=(True if sv is None else bool(sv)),
64                     batch_size=int(w.batch_size or 0),
65                     prefetch_batches=int(w.prefetch_batches or 0),
66                     decode_backend=env("WASB_DECODE_BACKEND") or w.decode_backend,
67                 )
68
69             @@ -203,6 +212,11 @@ class NativeWasbRunner(DetectorRunner):
70                 return False, (
71                     f"unknown device {self.cfg.device!r} ? valid: {'',
72                     '.join(VALID_DEVICES)})."
73                 )
74             + if self.cfg.decode_backend not in _VALID_DECODE_BACKENDS:
75             +     return False, (
76             +         f"unknown decode_backend {self.cfg.decode_backend!r} ? valid: "
77             +         f"{'', '.join(_VALID_DECODE_BACKENDS)})."
78             +     )
79             + try:
80             +     res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
81             + except FileNotFoundError:
82             +     @@ -231,6 +245,14 @@ class NativeWasbRunner(DetectorRunner):
83             +         "inference will be slow. Set indexing.wasb.device=cuda to require a
84             +         GPU.",
85             +         out,
86             +     )
87             + # NVDEC decode is CUDA-only: an auto?cpu fallback (or explicit cpu/mps, which
88             + # the
89             + # config validator already rejects) cannot run the GPU-resident path ? fail
90             + # loud
91             + # instead of letting wasb_infer.py die mid-run on a paid box.
92             + if self.cfg.decode_backend == "nvdec_resident" and ctx.effective != "cuda":
93             +     return False, (
94             +         f"decode_backend=nvdec_resident requires CUDA, but the effective device
95             +         is "
96             +         f"{ctx.effective!r} ({out}) ? NVDEC decode runs on the GPU."
97             +     )
98             + return True, out
99
100             def run_predict(
101                 @@ -263,7 +285,12 @@ class NativeWasbRunner(DetectorRunner):
102                     os.path.abspath(str(video_win_path)),
103                 ]
104                 frames_dir: Optional[str] = None
105                 - if self.cfg.stream_video:
106                 + if self.cfg.decode_backend == "nvdec_resident":
107                 +     # GPU-resident path (#506): wasb_infer.py decodes the video on the GPU

```

```

itself
102      +          # (PyNvVideoCodec NVDEC) ? no PNG extraction and no cv2 streaming, so
neither
103      +          # --stream-video nor --frames_out_dir applies and there is no frame cache
to clean.
104      +          argv += ["--decode-backend", self.cfg.decode_backend]
105      +      elif self.cfg.stream_video:
106          # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
107          argv.append("--stream-video")
108      else:
109          @@ -273,6 +300,15 @@ class NativeWasbRunner(DetectorRunner):
110              # M4: resolve device="auto" to the effective device (cuda-if-available-else-
cpu); an
111              # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to
wasb_infer.py.
112              device = self._effective_device()
113              +      if self.cfg.decode_backend == "nvdec_resident" and device != "cuda":
114              +          # Mirrors the healthcheck gate for callers that skipped it (e.g. auto?cpu
on a
115              +          # GPU-less box): NVDEC decode cannot run without CUDA ? fail before the
subprocess.
116              +          logger.error(
117              +              "decode_backend=nvdec_resident requires CUDA, but the effective device
is "
118              +              "%r ? aborting before the WASB subprocess.",
119              +              device,
120              +          )
121              +          return None
122          argv += [
123              "--weights",
124              weights,
125          diff --git a/backend/pipeline/detectors/wasb_infer.py
b/backend/pipeline/detectors/wasb_infer.py
126      index cbe1416..f41427b 100644
127      --- a/backend/pipeline/detectors/wasb_infer.py
128      +++ b/backend/pipeline/detectors/wasb_infer.py
129      @@ -30,7 +30,16 @@ resume mid-stream). We exploit that split:
130
131      The cache is keyed by the output path (`<out>_wasbcache/` by default), lives next
132      to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
133      -weights, sport, window size, or frame count change.
134      +weights, sport, window size, frame count, device, or decode backend change.
135      +
136      +GPU-resident decode (#506)
137      +-----
138      +`--decode-backend nvdec_resident` (default `cpu`) moves decode + resize + normalize
139      +onto the GPU with no host round-trip: PyNvVideoCodec NVDEC decode ? affine
140      +`grid_sample` resize replicating WASB's `cv2.warpAffine` exactly ? GPU ImageNet
141      +normalize ? detector. Validated on a real L4 (issue #506): ~1.7x the cv2 streaming
142      +path with rally F1 preserved on the hard golden clip. CUDA-only; needs the torch-2.x +
143      +PyNvVideoCodec env (the WSL torch-1.11 env keeps `cpu`, its behaviour untouched).
144
145      Usage (in WSL, from ~/models/WASB-SBDT/src):
146      python wasb_infer.py --frames_dir /path/frames --weights
../pretrained_weights/wasb_badminton_best.pth.tar \
147      @@ -53,6 +62,16 @@ CACHE_VERSION = 1
148      _DET_FILE = "det_raw.jsonl"
149      _MANIFEST_FILE = "manifest.json"
150
151      ## Pixel-decode backends (#506). "cpu" = the historical cv2 paths (disk frames or
152      ## --stream-video); "nvdec_resident" = GPU-resident NVDEC decode + affine grid_sample
153      ## resize + normalize (torch 2.x + PyNvVideoCodec, CUDA-only).
154      +DECODE_BACKENDS = ("cpu", "nvdec_resident")
155      +

```

```

156     ## ImageNet statistics ? MUST match WASB's build_img_transforms Normalize (the GPU-
resident
157     ## frontend normalizes on-device instead of through torchvision's transform).
158     +IMAGENET_MEAN = (0.485, 0.456, 0.406)
159     +IMAGENET_STD = (0.229, 0.224, 0.225)
160     +
161
162     def _build_cfg(weights: str, sport: str, device: str = "cuda"):
163         """Compose the WASB eval config via Hydra, overriding model/weights/device.
164     @@ -85,6 +104,48 @@ def _frame_id(path: str) -> int:
165         return int(digits) if digits else -1
166
167
168     ## ----- #
169     ## Runtime-compat shims for the torch-2.x / NumPy-2.x env (#506). Both are exact
170     ## no-ops on the verified WSL torch-1.11 + numpy<1.24 stack.
171     ## ----- #
172     +def _ensure_np_inf() -> None:
173     +     """NumPy 2.0 removed the ``np.Inf`` alias, which WASB's OnlineTracker still reads.
174     +     Restore it before any tracker use; a no-op on NumPy 1.x."""
175     +     import numpy as np
176     +
177     +     if not hasattr(np, "Inf"):
178     +         np.Inf = np.inf # type: ignore[attr-defined]
179     +
180     +
181     +@contextmanager
182     +def _torch_load_compat():
183     +     """Scope ``torch.load`` to ``weights_only=False`` while WASB builds its detector.
184     +
185     +     WASB's ``detector.py`` calls ``torch.load(model_path)`` with no ``weights_only``
186     +     argument; torch 2.6 flipped that default to True, which rejects the (pre-2.6,
187     +     owner-staged, trusted) WASB checkpoint. The kwarg is injected only when the
188     +     installed torch has it AND the caller didn't pass it, so torch 1.11 (no such
189     +     kwarg) and explicit callers are untouched."""
190     +     import inspect
191     +
192     +     import torch
193     +
194     +     real_load = torch.load
195     +     if "weights_only" not in inspect.signature(real_load).parameters:
196     +         yield # torch < 1.13 ? nothing to shim
197     +         return
198     +
199     +     def _load(*args, **kwargs):
200     +         kwargs.setdefault("weights_only", False)
201     +         return real_load(*args, **kwargs)
202     +
203     +     torch.load = _load
204     +     try:
205     +         yield
206     +     finally:
207     +         torch.load = real_load
208     +
209     +
210     # ----- #
211     # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
212     # ----- #
213     @@ -151,6 +212,7 @@ def manifest_compatible(
214         frames_in: int,
215         frames_total: int,
216         device: str = "cuda",
217     +     decode_backend: str = "cpu",
218     ) -> bool:
219         """A cache is reusable only if the run that produced it matches this run's inputs.

```

```

220
221     @@ -158,6 +220,11 @@ def manifest_compatible(
222         field) stay compatible with a CUDA run ? but a cpu/mps run will NOT reuse a cuda
cache
223         (detections can differ numerically across devices), and vice-versa.
224
225     +     ``decode_backend`` follows the same pattern (#506): pre-existing caches (no field)
226     +     are ``cpu``-decoded, so they stay compatible with cpu runs ? but an
``nvdec_resident``
227     +     run never reuses a cpu cache (NVDEC decode + the GPU affine warp are trajectory-
level,
228     +     not byte-level, equivalents), and vice-versa.
229     +
230     ``frames_dir`` is the **already-canonical cache key** the caller stored in the
manifest
231     (``run()`` passes its ``cache_key`` here, and writes the same value into the
manifest):
232     ``osp.abspath(frames_dir)`` for the frames-on-disk path, or the synthetic
``video:<abspath>``
233     @@ -174,6 +241,7 @@ def manifest_compatible(
234         and manifest.get("frames_in") == frames_in
235         and manifest.get("frames_total") == frames_total
236         and manifest.get("device", "cuda") == device
237     +     and manifest.get("decode_backend", "cpu") == decode_backend
238     )
239
240
241     @@ -487,17 +555,11 @@ def run(
242         from dataloaders import build_img_transforms, build_seq_transforms
243         from dataloaders.dataset_loader import ImageDataset
244         from torch.utils.data import DataLoader
245     -
246     -     # NB: this is WASB's OWN local `trackers` module (from its repo, on sys.path inside
the
247     -     # `wasb` conda env), NOT the Roboflow `trackers` package. The `type: ignore`
suppresses
248     -     # the mypy false positive where the main env resolves `trackers` to the Roboflow
package
249     -     # (which has no `build_tracker`). There is NO runtime collision ? this runs in a
separate
250     -     # subprocess/conda env. Do NOT "clean up" the ignore without removing the Roboflow
dep.
251     -     from trackers import build_tracker # type: ignore[attr-defined]
252     -     from utils import Center
253
254     -     from detectors import build_detector
255
256     +     _ensure_np_inf() # NumPy-2.x env: the tracker pass reads np.Inf
257     +     streaming = frame_loader is not None
258
259     -     cfg = _build_cfg(weights, sport, device)
260     @@ -550,6 +612,7 @@ def run(
261         frames_in=frames_in,
262         frames_total=len(frames),
263         device=device,
264     +     decode_backend="cpu", # run() IS the cpu decode path (disk frames / cv2
stream)
265     )
266     if resume:
267         windows_done, det_by_fid = load_and_clean_cache(cache_dir)
268     @@ -570,6 +633,7 @@ def run(
269         "frames_in": frames_in,
270         "frames_total": len(frames),
271         "device": device,
272     +     "decode_backend": "cpu",

```

```

273         "windows_total": windows_total,
274         "windows_done": windows_done,
275     }
276     @@ -598,7 +662,8 @@ def run(
277         loader = DataLoader(
278             ds, batch_size=batch_size, shuffle=False, num_workers=loader_workers
279         )
280         - detector = build_detector(cfg)
281         + with _torch_load_compat(): # torch-2.6 env: WASB's torch.load needs
weights_only=False
282         + detector = build_detector(cfg)
283
284         from contextlib import ExitStack
285
286     @@ -682,6 +747,25 @@ def run(
287         logger.info("detector cache already complete -> tracker-only re-run")
288
289         # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
290         + return _tracker_pass_and_write(cfg, det_by_fid, out_csv)
291         +
292         +
293     +def _tracker_pass_and_write(cfg, det_by_fid, out_csv: str) -> str:
294         + """Tracker re-run over the full ordered detection cache ? trajectory CSV.
295         +
296         + Shared verbatim by every decode backend: the tracker consumes the same per-frame
297         + candidate lists regardless of how the pixels were decoded, so keeping this single
298         + keeps the output contract single.
299         +
300         + NB: this is WASB's OWN local `trackers` module (from its repo, on sys.path inside
the
301         + `wasb` conda env), NOT the Roboflow `trackers` package. The `type: ignore`
suppresses
302         + the mypy false positive where the main env resolves `trackers` to the Roboflow
package
303         + (which has no `build_tracker`). There is NO runtime collision ? this runs in a
separate
304         + subprocess/conda env. Do NOT "clean up" the ignore without removing the Roboflow
dep.
305         + """
306         + from trackers import build_tracker # type: ignore[attr-defined]
307         +
308         + _ensure_np_inf() # OnlineTracker reads np.Inf (removed in NumPy 2.0)
309         + tracker = build_tracker(cfg)
310         + tracker.refresh()
311         + rows = []
312     @@ -832,6 +916,320 @@ def run_video_streaming(
313         release()
314
315
316     +## ----- #
317     +## GPU-resident decode backend (#506): NVDEC decode ? affine grid_sample resize ?
318     +## normalize ? detector, all on the GPU (no host round-trip). Validated on a real
319     +## L4: ~1.7x the cv2 streaming path with rally F1 preserved (0.574 ? baseline 0.582
320     +## on GX010128 at the SERVED preset).
321     +## ----- #
322     +def build_affine_grid(trans_inv, inp_w: int, inp_h: int, orig_w: int, orig_h: int):
323         + """`F.grid_sample` sampling grid replicating `cv2.warpAffine(img, trans,
324         + (inp_w, inp_h), INTER_LINEAR)` ? THE #506 lesson lives here.
325         +
326         + The GPU resize MUST be WASB's affine warp (center/scale geometry + cv2's
327         + integer-coordinates-are-pixel-centers convention), NOT a plain resize:
328         + `F.interpolate`'s half-pixel mapping shifts every sample by a constant
329         + sub-pixel offset, which tips the sigmoid past `score_threshold` on marginal
330         + frames and silently over-detects ? F1 0.58 ? 0.07 on the hard clip GX010128,
331         + while the easy clip masked it at 98% agreement. Locked by a cv2-parity unit test.

```

```

332 +
333 + ``trans_inv`` is WASB's INVERSE affine (``get_affine_transform(..., inv=1)``,
334 + mapping output px ? source px; each source position is normalized to [-1, 1]
335 + via ``gx = (2*x + 1)/W - 1`` (cv2's half-pixel convention). Pure torch+numpy ?
336 + no WASB imports ? so the parity test runs on any CPU box.
337 +
338 + Returns a ``[1, inp_h, inp_w, 2]`` float32 grid for ``F.grid_sample(...,
339 + mode='bilinear', align_corners=False, padding_mode='border')``.
340 + """
341 + import numpy as np
342 + import torch
343 +
344 + ys, xs = torch.meshgrid(
345 +     torch.arange(inp_h, dtype=torch.float32),
346 +     torch.arange(inp_w, dtype=torch.float32),
347 +     indexing="ij",
348 + )
349 + out_coords = torch.stack([xs, ys, torch.ones_like(xs)], dim=-1) # [H,W,3] output
px
350 + mt = torch.from_numpy(np.ascontiguousarray(trans_inv, dtype=np.float32)) # [2,3]
351 + src = out_coords @ mt.transpose(0, 1) # [H,W,2] source px
352 + gx = (2.0 * src[..., 0] + 1.0) / orig_w - 1.0
353 + gy = (2.0 * src[..., 1] + 1.0) / orig_h - 1.0
354 + return torch.stack([gx, gy], dim=-1).unsqueeze(0) # [1,H,W,2]
355 +
356 +
357 +def _wasb_inverse_affines(orig_h: int, orig_w: int, out_wh, out_scales):
358 +     """Per-scale inverse affines exactly as WASB's ``ImageDataset`` builds
359 +     ``trans_outputs_inv`` (``get_transform(img, wh, inv=1)``: center = frame middle,
360 +     scale = max(h, w), output size halved per extra scale). Returns
361 +     ``{scale: np.ndarray[2, 3] float64}`` ? float64 like ``cv2.getAffineTransform``,
362 +     matching what the DataLoader path collates for the postprocessor."""
363 +     import numpy as np
364 +     from utils.image import get_affine_transform
365 +
366 +     c = np.array([orig_w / 2.0, orig_h / 2.0], dtype=np.float32)
367 +     s = max(orig_h, orig_w) * 1.0
368 +     out = {}
369 +     out_w, out_h = int(out_wh[0]), int(out_wh[1])
370 +     for scale in out_scales:
371 +         out[scale] = get_affine_transform(c, s, 0, [out_w, out_h], inv=1)
372 +         out_w //= 2
373 +         out_h //= 2
374 +     return out
375 +
376 +
377 +class NvdecResidentFrontend:
378 +     """On-GPU frame source for the ``nvdec_resident`` detector pass (#506).
379 +
380 +     PyNvVideoCodec ``SimpleDecoder(output_color_type=RGB)`` decodes on the GPU's
381 +     NVDEC engine; frames arrive as CUDA tensors via DLPack and never touch the host.
382 +     ``read_chw`` then applies the cv2-faithful affine warp (``build_affine_grid``)
383 +     and ImageNet-normalizes on-device, yielding the same ``[3, inp_h, inp_w]``
384 +     float32 tensor per frame that the ImageDataset path assembles on the CPU.
385 +     """
386 +
387 +     def __init__(self, video_path: str, inp_wh, gpu_id: int = 0):
388 +         import PyNvVideoCodec as nvc
389 +         import torch
390 +
391 +         self._torch = torch
392 +         self._device = torch.device("cuda", gpu_id)
393 +         inp_w, inp_h = int(inp_wh[0]), int(inp_wh[1])
394 +         try:
395 +             self._dec = nvc.SimpleDecoder(

```



```

396 +         video_path, gpu_id=gpu_id, output_color_type=nvc.OutputColorType.RGB
397 +     )
398 +     except Exception as e:
399 +         raise RuntimeError(
400 +             f"PyNvVideoCodec RGB SimpleDecoder failed for {video_path}: {e!r} ? "
401 +             f"decode_backend=nvdec_resident needs PyNvVideoCodec with RGB output "
402 +             f"and libnvcuvid/libnvidia-encode on the box
(deploy/cloudrun/Dockerfile "
403 +             f"/ gpu_setup.sh stage them). Fall back to decode_backend=cpu."
404 +         ) from e
405 +     self.count = len(self._dec)
406 +     probe = self._decode_rgb_hwc(0)
407 +     self.orig_h, self.orig_w = int(probe.shape[0]), int(probe.shape[1])
408 +     trans_inp_inv = _wasb_inverse_affines(
409 +         self.orig_h, self.orig_w, (inp_w, inp_h), [0]
410 +     )[0]
411 +     self._grid = build_affine_grid(
412 +         trans_inp_inv, inp_w, inp_h, self.orig_w, self.orig_h
413 +     ).to(self._device)
414 +     self._mean = torch.tensor(IMAGENET_MEAN, device=self._device).view(3, 1, 1)
415 +     self._std = torch.tensor(IMAGENET_STD, device=self._device).view(3, 1, 1)
416 +
417 +     def _decode_rgb_hwc(self, index: int):
418 +         """Frame ``index`` as an on-GPU HWC RGB uint8 tensor (zero-copy via DLPack)."""
419 +         torch = self._torch
420 +         t = torch.from_dlpack(self._dec[index])
421 +         if t.dim() == 3 and t.shape[0] == 3 and t.shape[-1] != 3:
422 +             t = t.permute(1, 2, 0) # CHW layout variant ? HWC
423 +         if t.dim() != 3 or t.shape[-1] != 3:
424 +             raise RuntimeError(
425 +                 f"unexpected NVDEC frame shape {tuple(t.shape)} for frame {index} ? "
426 +                 f"expected 3-channel RGB (is output_color_type=RGB supported here?)"
427 +             )
428 +         return t
429 +
430 +     def read_chw(self, index: int):
431 +         """Decoded frame ``index`` ? the normalized ``[3, inp_h, inp_w]`` float32 CUDA
432 +         tensor the detector expects (warpAffine-faithful resize + ImageNet
normalize)."""
433 +         import torch.nn.functional as F
434 +
435 +         hwc = self._decode_rgb_hwc(index)
436 +         chw = hwc.permute(2, 0, 1).unsqueeze(0).float() / 255.0
437 +         warped = F.grid_sample(
438 +             chw, self._grid, mode="bilinear", align_corners=False,
padding_mode="border"
439 +         ).squeeze(0)
440 +         return (warped - self._mean) / self._std
441 +
442 +     def release(self):
443 +         self._dec = None
444 +
445 +
446 +     def run_video_nvdec_resident(
447 +         video_path: str,
448 +         weights: str,
449 +         out_csv: str,
450 +         sport: str = "badminton",
451 +         limit: int = 0,
452 +         batch_size: int = 8,
453 +         log_every_batches: int = 50,
454 +         cache_dir: "str | None" = None,
455 +         flush_every: int = 1000,
456 +         fresh: bool = False,
457 +     ) -> str:

```

```

458 + """GPU-resident path (#506): NVDEC-decode ``video_path`` on the GPU and feed
459 + warped+normalized frames straight to the detector ? decode, resize, and
460 + normalize never touch the host. CUDA-only by construction.
461 +
462 + NOT byte-identical to the ``cpu`` decode paths (NVDEC's YUV?RGB and the GPU
463 + warp differ from cv2 at float precision) ? equivalence is trajectory-level
464 + (the #506 unit gate: 96.3% visibility agreement, median 0.036 px) and
465 + rally-F1-level (the golden gate), so the detection cache is keyed by
466 + ``decode_backend`` and never shared with cpu-decoded runs. Same resumable
467 + cache + tracker tail as ``run()`` otherwise.
468 + """
469 + import time
470 +
471 + import numpy as np
472 + import torch
473 +
474 + from detectors import build_detector
475 +
476 + _ensure_np_inf()
477 + if not torch.cuda.is_available():
478 +     raise SystemExit("decode_backend=nvdec_resident requires a CUDA GPU")
479 +
480 + cfg = _build_cfg(weights, sport, "cuda")
481 + frames_in = int(cfg["model"]["frames_in"])
482 + input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
483 + output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
484 + out_scales = list(cfg["model"]["out_scales"])
485 +
486 + fe = NvdecResidentFrontend(video_path, input_wh)
487 + try:
488 +     total = min(fe.count, limit) if limit else fe.count
489 +     if total < frames_in:
490 +         raise SystemExit(f"need >= {frames_in} frames, video has {total}")
491 +     windows_total = total - frames_in + 1
492 +     logger.info(
493 +         f"nvdec_resident: {total} frames ({fe.orig_w}x{fe.orig_h}), "
494 +         f"{windows_total} windows"
495 +     )
496 +
497 +     # ---- cache setup / resume decision (mirrors run()); keyed nvdec_resident) ---
498 +     #
499 +     if cache_dir is None:
500 +         cache_dir = default_cache_dir(out_csv)
501 +         os.makedirs(cache_dir, exist_ok=True)
502 +         det_jsonl, _ = _cache_paths(cache_dir)
503 +         cache_key = "video:" + osp.abspath(video_path)
504 +         manifest = None if fresh else read_manifest(cache_dir)
505 +         resume = manifest_compatible(
506 +             manifest or {},
507 +             frames_dir=cache_key,
508 +             weights=weights,
509 +             sport=sport,
510 +             frames_in=frames_in,
511 +             frames_total=total,
512 +             device="cuda",
513 +             decode_backend="nvdec_resident",
514 +         )
515 +         if resume:
516 +             windows_done, det_by_fid = load_and_clean_cache(cache_dir)
517 +             logger.info(
518 +                 f"resuming from cache: {windows_done}/{windows_total} windows already
519 +                 detected"
520 +             )
521 +         else:
522 +             if manifest is not None:

```

```

521 +         logger.info("cache incompatible with this run -> starting fresh")
522 +         reset_cache(cache_dir)
523 +         windows_done, det_by_fid = 0, defaultdict(list)
524 +
525 +     base_manifest = {
526 +         "version": CACHE_VERSION,
527 +         "frames_dir": cache_key,
528 +         "weights": weights,
529 +         "sport": sport,
530 +         "frames_in": frames_in,
531 +         "frames_total": total,
532 +         "device": "cuda",
533 +         "decode_backend": "nvdec_resident",
534 +         "windows_total": windows_total,
535 +         "windows_done": windows_done,
536 +     }
537 +     write_manifest(cache_dir, base_manifest)
538 +
539 +     # ---- GPU-resident detector pass over the un-cached windows -----
540 +     #
541 +     if windows_done < windows_total:
542 +         with _torch_load_compat():
543 +             detector = build_detector(cfg)
544 +             # {scale: [1,2,3] float64 CPU tensor} ? the postprocessor .cpu().numpy()'s
545 +             # these, so CPU float64 matches the DataLoader-collated path exactly.
546 +             trans_t = {
547 +                 s: torch.from_numpy(np.ascontiguousarray(m)).unsqueeze(0)
548 +                 for s, m in _wasb_inverse_affines(
549 +                     fe.orig_h, fe.orig_w, output_wh, out_scales
550 +                 ).items()
551 +             }
552 +             t0 = time.time()
553 +             done = windows_done
554 +             win_idx = windows_done
555 +             n_batches = 0
556 +             log_every = max(1, log_every_batches)
557 +             flush_n = max(1, flush_every)
558 +             pending = []
559 +             window = [] # sliding [(fid, chw)] of the last frames_in decoded frames
560 +             batch_fids, batch_imgs = [], []
561 +             logger.info(
562 +                 f"detecting on {windows_total - windows_done} remaining windows "
563 +                 f"(total {windows_total}, batch={batch_size}, source=nvdec-
564 +                 resident)..."
565 +             )
566 +             det_f = open(det_jsonl, "a")
567 +             try:
568 +                 with torch.no_grad():
569 +                     # Window i needs frames [i, i+frames_in) ? start decoding at the
570 +                     # first un-cached window's first frame (NVDEC seeks past the
571 +                     # prefix).
572 +                     for idx in range(windows_done, total):
573 +                         window.append((idx, fe.read_chw(idx)))
574 +                         if len(window) > frames_in:
575 +                             window.pop(0)
576 +                         if len(window) < frames_in:
577 +                             continue
578 +                         batch_fids.append([fid for fid, _ in window])
579 +                         batch_imgs.append(
580 +                             torch.cat([chw for _, chw in window], dim=0)
581 +                         )
582 +                         if len(batch_imgs) < batch_size and idx < total - 1:
583 +                             continue
584 +                         nb = len(batch_imgs)
585 +                         imgs = torch.stack(batch_imgs, dim=0)

```

```

583 + # .repeat (not .expand): the postprocessor calls .numpy() on
these.
584 +
585 + affine_mats = {
586 +     s: m.repeat(nb, 1, 1) for s, m in trans_t.items()
587 + }
588 + batch_results, _ = detector.run_tensor(imgs, affine_mats)
589 + for ib in range(nb):
590 +     frames_payload = []
591 +     for ie in sorted(batch_results[ib].keys()):
592 +         fid = batch_fids[ib][ie]
593 +         plain = [_det_plain(d) for d in batch_results[ib][ie]]
594 +         frames_payload.append([fid, plain])
595 +         det_by_fid[fid].extend(plain)
596 +         pending.append({"w": win_idx, "f": frames_payload})
597 +         win_idx += 1
598 + done += nb
599 + n_batches += 1
600 + batch_fids, batch_imgs = [], []
601 + if len(pending) >= flush_n or done >= windows_total:
602 +     _append_windows(det_f, pending)
603 +     pending = []
604 +     base_manifest["windows_done"] = done
605 +     write_manifest(cache_dir, base_manifest)
606 + if n_batches % log_every == 0 or done >= windows_total:
607 +     el = time.time() - t0
608 +     new_done = done - windows_done
609 +     rate = new_done / el if el > 0 else 0.0
610 +     eta = (windows_total - done) / rate if rate > 0 else 0.0
611 +     logger.info(
612 +         f"{done}/{windows_total} "
613 +         f"({100 * done // max(1, windows_total)}%) | {rate:.1f}
win/s | "
614 +         f"elapsed {el:.0f}s | ETA {eta:.0f}s"
615 +     )
616 + if pending: # safety net; the done>=windows_total branch already
flushed
617 +     _append_windows(det_f, pending)
618 +     base_manifest["windows_done"] = done
619 +     write_manifest(cache_dir, base_manifest)
620 + finally:
621 +     det_f.close()
622 +     logger.info(f"detection done in {time.time() - t0:.0f}s; running
tracker...")
623 + else:
624 +     logger.info("detector cache already complete -> tracker-only re-run")
625 + finally:
626 +     fe.release()
627 + return _tracker_pass_and_write(cfg, det_by_fid, out_csv)
628 +
629 +
630 def main():
631     ap = argparse.ArgumentParser()
632     ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
633     @@ -847,6 +1245,16 @@ def main():
634         help="FAST PATH: decode --video on the fly and feed frames straight to "
635         "the detector (no PNG extraction). Output-equivalent to the disk path.",
636     )
637 + ap.add_argument(
638 +     "--decode-backend",
639 +     default="cpu",
640 +     choices=list(DECODE_BACKENDS),
641 +     help="cpu (default) = the historical cv2 decode paths (disk frames or "
642 +     "--stream-video). nvdec_resident = GPU-RESIDENT decode+resize+normalize "
643 +     "(#506): PyNvVideoCodec NVDEC + cv2-faithful affine grid_sample, CUDA-only, "

```

```

644 +         "~1.7x on an L4. Requires --video; detections cache separately from cpu "
645 +         "(trajectory-level, not byte-level, equivalence).",
646 +     )
647     ap.add_argument("--weights", required=True)
648     ap.add_argument("--out", required=True)
649     ap.add_argument("--sport", default="badminton")
650 @@ -887,6 +1295,30 @@ def main():
651 )
652     args = ap.parse_args()
653
654 +     if args.decode_backend == "nvdec_resident":
655 +         # GPU-resident branch FIRST: it decodes the video itself (NVDEC), so neither
the
656 +         # --stream-video cv2 path nor the extract-frames fallback below may run.
657 +         if not args.video:
658 +             raise SystemExit("--decode-backend nvdec_resident requires --video")
659 +         if args.device != "cuda":
660 +             raise SystemExit(
661 +                 "--decode-backend nvdec_resident requires --device cuda "
662 +                 "(NVDEC decode runs on the GPU)"
663 +             )
664 +         run_video_nvdec_resident(
665 +             args.video,
666 +             args.weights,
667 +             args.out,
668 +             args.sport,
669 +             args.limit,
670 +             batch_size=args.batch_size,
671 +             log_every_batches=args.log_every_batches,
672 +             cache_dir=args.cache_dir,
673 +             flush_every=args.flush_every,
674 +             fresh=args.fresh,
675 +         )
676 +         return
677 +
678     if args.stream_video:
679         if not args.video:
680             raise SystemExit("--stream-video requires --video")
681 diff --git a/deploy/cloudrun/Dockerfile b/deploy/cloudrun/Dockerfile
682 index 836ffb7..aee76a0 100644
683 --- a/deploy/cloudrun/Dockerfile
684 +++ b/deploy/cloudrun/Dockerfile
685 @@ -8,13 +8,33 @@
686     # Two python stacks, deliberately separate (a framework conflict otherwise ? same split
as the
687     # native GPU box, deploy/digitalocean/gpu_setup.sh):
688     # * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
689     -# * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as
a subprocess
690     -# (the native runner shells out to $WASB_PYTHON), so the app's torch never co-
installs with it.
691     +# * a conda `wasb` env (py3.10 + torch 2.6.0+cu124 + PyNvVideoCodec ? the #506
modernized stack,
692     +# validated on a real L4: cpu-decode F1 0.576 ? the torch-1.11 baseline 0.582) runs
the WASB
693     +# shuttle detector as a subprocess (the native runner shells out to $WASB_PYTHON),
so the app's
694     +# torch never co-installs with it.
695     #
696     # ? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance
NOT cleared for
697     # commercial sharing. They are NOT baked into this image; stage them at runtime (a
bucket fetch or a
698     # mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before any public, non-
owner serving.

```

```

699
700     ## Driver userspace libs for NVDEC/NVENC (#506): GPU hosts inject the COMPUTE driver
stack but not
701     ## the video-codec libs ? no libnvcuvid.so.1, no libnvidia-encode.so.1 ? and
PyNvVideoCodec needs
702     ## BOTH (it initializes NVENC even for decode-only). Extract the pair from the matching
datacenter
703     ## driver .run in a throwaway stage. Keep NVIDIA_DRIVER_VERSION aligned with the GPU
fleet's driver
704     ## (580.159.03 = the validated L4 combo); these userspace libs talk to the host-injected
libcuda,
705     ## so a large version skew can break decode ? the golden gate on the real image is the
check.
706     +ARG NVIDIA_DRIVER_VERSION=580.159.03
707     +FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04 AS nvlibs
708     +ARG NVIDIA_DRIVER_VERSION
709     +RUN apt-get update && apt-get install -y --no-install-recommends curl ca-certificates \
710     +     && rm -rf /var/lib/apt/lists/* \
711     +     && curl -fsSL -o /tmp/drv.run \
712     +         "https://us.download.nvidia.com/tesla/${NVIDIA_DRIVER_VERSION}/NVIDIA-
Linux-x86_64-${NVIDIA_DRIVER_VERSION}.run" \
713     +     && sh /tmp/drv.run --extract-only --target /tmp/drv \
714     +     && mkdir -p /nvlibs \
715     +     && cp -a /tmp/drv/libnvcuvid.so.* /tmp/drv/libnvidia-encode.so.* /nvlibs/ \
716     +     && rm -rf /tmp/drv /tmp/drv.run
717     +
718     FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04
719
720     ENV DEBIAN_FRONTEND=noninteractive \
721     @@ -29,22 +49,38 @@ RUN apt-get update && apt-get install -y --no-install-recommends \
722         python3 python3-pip python3-venv \
723         && rm -rf /var/lib/apt/lists/*
724
725     ## NVDEC/NVENC userspace libs (#506, from the nvlibs stage above) + the .so.1 sonames
726     ## PyNvVideoCodec dlopens. Harmless when decode_backend=cpu; required for
nvdec_resident.
727     +ARG NVIDIA_DRIVER_VERSION
728     +COPY --from=nvlibs /nvlibs/ /usr/lib/x86_64-linux-gnu/
729     +RUN ln -sf "libnvcuvid.so.${NVIDIA_DRIVER_VERSION}" /usr/lib/x86_64-linux-
gnu/libnvcuvid.so.1 \
730     +     && ln -sf "libnvidia-encode.so.${NVIDIA_DRIVER_VERSION}" /usr/lib/x86_64-linux-
gnu/libnvidia-encode.so.1 \
731     +     && ldconfig
732     +
733     # --- WASB detector env (micromamba + conda-forge ? BSD-licensed, NO Anaconda ToS; P3a,
ADR-005) ---
734     # Enforces ADR-005 (every dependency ? code, data, weights, AND the build toolchain ?
stays
735     # MIT/Apache/BSD): micromamba (BSD-3) + conda-forge (community) replaces Miniconda's
Anaconda-ToS-gated
736     # `defaults` channels (the ?200-employee Terms-of-Service trap flagged in
PACKAGING_PLAN.md §3).
737     # MAMBA_ROOT_PREFIX=$CONDA_DIR (= /opt/conda) keeps the env at $CONDA_DIR/envs/wasb so
$WASB_PYTHON below
738     -# is UNCHANGED. Same verified torch-1.11.0+cu113 WASB stack; the pip wheels are
unaffected by the switch.
739     ## is UNCHANGED.
740     ##
741     ## #506 modernized stack (validated end-to-end on a real L4, 2026-07-06): py3.10 + torch
2.6.0+cu124
742     ## (torch ?2 needs py?3.9; WASB HRNet ports clean ? cpu F1 0.576 ? the torch-1.11
baseline 0.582) +
743     ## PyNvVideoCodec (MIT ? the practical NVDEC path; TorchCodec needs a matching ffmpeg
build and could
744     ## not load) for decode_backend=nvdec_resident. wasb_infer.py carries the two version

```

```

shims this bump
745     ## needs (torch.load weights_only=False; the NumPy-2.0 np.Inf alias for WASB's tracker).
numpy is
746     ## UNPINNED (the old "numpy<1.24" was the torch-1.11 ABI pin). WASB-SBDT ships no
requirements.txt ?
747     ## the explicit list below is the (L4-validated) dependency closure its src/ imports
need.
748     ENV MAMBA_ROOT_PREFIX=/opt/conda
749     RUN curl -sSL https://micro.mamba.pm/api/micromamba/linux-64/latest \
750         | tar -xj -C /usr/local bin/micromamba \
751     -     && micromamba create -y -q -n wasb -c conda-forge python=3.8 \
752     +     && micromamba create -y -q -n wasb -c conda-forge python=3.10 \
753         && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
754         && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
755     -         torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-index-url
https://download.pytorch.org/whl/cu113 \
756     -     && ( [ -f "$WASB_REPO_DIR/requirements.txt" ] && "$CONDA_DIR/envs/wasb/bin/python"
-m pip install -q -r "$WASB_REPO_DIR/requirements.txt" || true ) \
757     +         torch==2.6.0 torchvision==0.21.0 --index-url
https://download.pytorch.org/whl/cu124 \
758     +     && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q PyNvVideoCodec \
759         && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
760     -         hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml
matplotlib "numpy<1.24"
761     +         hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml
matplotlib numpy
762
763     # --- Main application env ---
764     WORKDIR /app
765     diff --git a/deploy/digitalocean/gpu_setup.sh b/deploy/digitalocean/gpu_setup.sh
766     index 8d47e49..a89cc1b 100755
767     --- a/deploy/digitalocean/gpu_setup.sh
768     +++ b/deploy/digitalocean/gpu_setup.sh
769     @@ -3,8 +3,14 @@
770     # Native (no-WSL) WASB shuttle-detector setup for a Linux GPU box ? the M3.5 enabler
for
771     # real trajectory extraction. Replicates docs/TRACKNET_WSL_SETUP.md's WSL conda env
772     # *natively* on Linux. Run AFTER ./bootstrap.sh --gpu (which provides CUDA torch for
the
773     -# main venv); this builds a SEPARATE conda env for WASB (torch 1.11) ? never co-
installed
774     -# with the indexer's torch (framework conflict).
775     ## main venv); this builds a SEPARATE conda env for WASB ? never co-installed with the
776     ## indexer's torch (framework conflict).
777     ##
778     ## #506 modernized stack (validated end-to-end on a real L4, 2026-07-06): py3.10 +
779     ## torch 2.6.0+cu124 (WASB HRNet ports clean; cpu F1 ? the torch-1.11 baseline) +
780     ## PyNvVideoCodec for the GPU-resident decode backend (indexing.wasb.decode_backend=
781     ## nvdec_resident, ~1.7x). Step [2/6] stages the NVDEC/NVENC driver userspace libs
782     ## PyNvVideoCodec needs (compute-only driver images ship neither).
783     #
784     # Requires an NVIDIA GPU + driver (the DO `gpu-*-base` image ships it).
785     #
786     @@ -18,7 +24,7 @@ CONDA_DIR="${CONDA_DIR:-$HOME/miniconda3}"
787     WASB_REPO="$MODELS_DIR/WASB-SBDT"
788     mkdir -p "$MODELS_DIR"
789
790     -echo "[gpu] [1/5] miniconda (native Linux)"
791     +echo "[gpu] [1/6] miniconda (native Linux)"
792     if [ ! -x "$CONDA_DIR/bin/conda" ]; then
793         curl -sSL https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -o
/tmp/mc.sh
794         bash /tmp/mc.sh -b -p "$CONDA_DIR" && rm -f /tmp/mc.sh
795     @@ -26,29 +32,49 @@ fi
796     # shellcheck disable=SC1091

```

```

797     source "$CONDA_DIR/etc/profile.d/conda.sh"
798
799     +echo "[gpu] [2/6] NVDEC/NVENC driver userspace libs (#506: PyNvVideoCodec needs BOTH)"
800     + # Compute-only driver images (GCE DLVM, DO gpu-*-base) ship neither libnvcuvid.so.1 nor
801     + # libnvidia-encode.so.1. Extract them from the .run matching the RUNNING driver so the
802     + # userspace/kernel versions can never skew (see the #506 / gpu-vm-setup-gotchas notes).
803     + if [ ! -f /usr/lib/x86_64-linux-gnu/libnvcuvid.so.1 ] || [ ! -f /usr/lib/x86_64-linux-
804     +   gnu/libnvidia-encode.so.1 ]; then
805     +   DRV="$(nvidia-smi --query-gpu=driver_version --format=csv,noheader | head -1)"
806     +   echo "[gpu]          extracting libnvcuvid + libnvidia-encode from driver $DRV ..."
807     +   curl -fsSL -o /tmp/drv.run "https://us.download.nvidia.com/tesla/$DRV/NVIDIA-
808     +   Linux-x86_64-$DRV.run" \
809     +   || curl -fsSL -o /tmp/drv.run
810     +   "https://us.download.nvidia.com/XFree86/Linux-x86_64/$DRV/NVIDIA-Linux-x86_64-$DRV.run"
811     +   sh /tmp/drv.run --extract-only --target /tmp/drv-extract >/dev/null
812     +   sudo cp -a /tmp/drv-extract/libnvcuvid.so.* /tmp/drv-extract/libnvidia-encode.so.*
813     +   /usr/lib/x86_64-linux-gnu/
814     +   ( cd /usr/lib/x86_64-linux-gnu \
815     +     && sudo ln -sf "libnvcuvid.so.$DRV" libnvcuvid.so.1 \
816     +     && sudo ln -sf "libnvidia-encode.so.$DRV" libnvidia-encode.so.1 \
817     +     && sudo ldconfig )
818     +   rm -rf /tmp/drv-extract /tmp/drv.run
819     + else
820     +   echo "[gpu]          already present"
821     + fi
822
823     # Modern conda BLOCKS non-interactive `conda create` until the defaults-channel Terms
824     of Service
825     # are accepted (CondaToSNonInteractiveError). Accept them so the env build doesn't fail
826     on a fresh
827     # box. (Observed on a GCP Deep-Learning-VM image, 2026-06-20.)
828     conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/main
829     >/dev/null 2>&1 || true
830     conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/r
831     >/dev/null 2>&1 || true
832
833     -echo "[gpu] [2/5] WASB-SBDT repo"
834     +echo "[gpu] [3/6] WASB-SBDT repo"
835     if [ ! -d "$WASB_REPO/.git" ]; then
836     git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO"
837     fi
838
839     -echo "[gpu] [3/5] conda env 'wasb' (py3.8 + torch 1.11.0+cu113 ? the verified WASB
840     stack)"
841     -conda env list | grep -q '/envs/wasb$' || conda create -y -q -n wasb python=3.8
842     +echo "[gpu] [4/6] conda env 'wasb' (py3.10 + torch 2.6.0+cu124 + PyNvVideoCodec ? #506
843     stack)"
844     +conda env list | grep -q '/envs/wasb$' || conda create -y -q -n wasb python=3.10
845     conda activate wasb
846     -python -m pip install -q torch==1.11.0+cu113 torchvision==0.12.0+cu113 \
847     - --extra-index-url https://download.pytorch.org/whl/cu113
848     -[ -f "$WASB_REPO/requirements.txt" ] && python -m pip install -q -r
849     "$WASB_REPO/requirements.txt" || true
850     -# WASB-SBDT's requirements.txt is INCOMPLETE: src/ imports hydra, pandas, etc. that it
851     omits, so
852     -# `import wasb_infer` deps fail without these. (Found on the GCP L4 port, 2026-06-20.)
853     numpy is
854     -# pinned <1.24 for torch-1.11 ABI compatibility.
855     -python -m pip install -q hydra-core omegaconf pandas opencv-python-headless pillow
856     scipy tqdm pyyaml matplotlib "numpy<1.24" || true
857     +python -m pip install -q torch==2.6.0 torchvision==0.21.0 \
858     + --index-url https://download.pytorch.org/whl/cu124
859     +python -m pip install -q PyNvVideoCodec
860     + # WASB-SBDT ships NO requirements.txt: src/ imports hydra, pandas, etc. ? this explicit
861     list is

```



```

847     ## the L4-validated dependency closure (#506). numpy is UNPINNED (the old "numpy<1.24"
was the
848     ## torch-1.11 ABI pin); wasb_infer.py shims the NumPy-2.0 np.Inf removal for WASB's
tracker.
849     +python -m pip install -q hydra-core omegaconf pandas opencv-python-headless pillow
scipy tqdm pyyaml matplotlib numpy || true
850
851     -echo "[gpu] [4/5] pretrained badminton weights"
852     +echo "[gpu] [5/6] pretrained badminton weights"
853     WEIGHTS="$WASB_REPO/pretrained_weights/wasb_badminton_best.pth.tar"
854     if [ -f "$WEIGHTS" ]; then
855         echo "[gpu]         found $WEIGHTS"
856     @@ -56,13 +82,18 @@ else
857         echo "[gpu]         !! MISSING $WEIGHTS ? fetch wasb_badminton_best.pth.tar
(Drive/bucket) before extraction."
858     fi
859
860     -echo "[gpu] [5/5] GPU sanity check (in the wasb env)"
861     +echo "[gpu] [6/6] GPU sanity check (in the wasb env)"
862     python - <<'PY'
863     import torch
864     ok = torch.cuda.is_available()
865     print("[gpu]         torch", torch.__version__, "| cuda_available", ok,
866           "| device", (torch.cuda.get_device_name(0) if ok else "CPU"))
867     assert ok, "CUDA not available in the wasb env ? check the GPU driver / base image"
868     +try:
869     +     import PyNvVideoCodec as pncv
870     +     print("[gpu]         PyNvVideoCodec", getattr(pncv, "__version__", "?"), "OK")
871     +except Exception as e: # nvdec_resident unavailable; the cpu decode path still works
872     +     print("[gpu]         !! PyNvVideoCodec import failed:", repr(e)[:150])
873     PY
874
875     echo "[gpu] native WASB env ready.  Activate with:  source
$CONDA_DIR/etc/profile.d/conda.sh && conda activate wasb"

```

3. user (2026-07-07T04:01:38.037Z)

```

1     """GPU-resident NVDEC decode backend (#506) ? the parts testable off-GPU, plus the
2     on-box parity gate.
3
4     THE lesson this file locks (issue #506): the GPU resize MUST replicate WASB's
5     ``cv2.warpAffine(get_affine_transform(...), INTER_LINEAR)`` exactly. A naive
6     ``F.interpolate`` resize samples at half-pixel-convention positions ? a constant
7     sub-pixel shift versus cv2's integer-pixel-center affine ? which tips the sigmoid
8     past ``score_threshold`` on marginal frames and silently over-detects (rally F1
9     0.58 ? 0.07 on the hard golden clip GX010128, while the easy clip masked it at 98%
10    trajectory agreement). The grid-parity tests below fail if anyone "simplifies" the
11    affine ``grid_sample`` back to a plain resize.
12
13    The trajectory-parity contract itself (``nvdec_resident`` ? ``cpu`` on a real clip)
14    is a HARDWARE test ? it needs a CUDA GPU, PyNvVideoCodec, the WASB repo, weights and
15    a fixture clip ? so it skips everywhere else (CI is CPU-only). See
16    ``test_nvdec_resident_trajectory_matches_cpu_on_fixture`` for how to run it on the box.
17    """
18
19    import csv
20    import importlib.util
21    import os
22    import os.path as osp
23    import sys
24
25    import numpy as np
26    import pytest
27
28    from backend.pipeline.detectors import wasb_infer as wi

```

```

29
30
31 # ----- #
32 # WASB's exact affine geometry, replicated for the tests (utils/image.py
33 # ``get_affine_transform`` with rot=0: center = frame middle, scale = max(h, w),
34 # 3-point correspondence through cv2.getAffineTransform). The runtime path calls
35 # WASB's own function; the tests only need matrices of the same shape.
36 # ----- #
37 def _get_3rd_point(a, b):
38     direct = a - b
39     return b + np.array([-direct[1], direct[0]], dtype=np.float32)
40
41
42 def _wasb_affine_matrices(orig_w, orig_h, dst_w, dst_h):
43     """(forward, inverse) 2x3 matrices exactly as WASB's get_transform builds them."""
44     import cv2
45
46     c = np.array([orig_w / 2.0, orig_h / 2.0], dtype=np.float32)
47     s = max(orig_h, orig_w) * 1.0
48     src = np.zeros((3, 2), dtype=np.float32)
49     dst = np.zeros((3, 2), dtype=np.float32)
50     src[0, :] = c
51     src[1, :] = c + np.array([0, -s * 0.5], dtype=np.float32)
52     dst[0, :] = [dst_w * 0.5, dst_h * 0.5]
53     dst[1, :] = dst[0, :] + np.array([0, -dst_w * 0.5], dtype=np.float32)
54     src[2, :] = _get_3rd_point(src[0, :], src[1, :])
55     dst[2, :] = _get_3rd_point(dst[0, :], dst[1, :])
56     fwd = cv2.getAffineTransform(np.float32(src), np.float32(dst))
57     inv = cv2.getAffineTransform(np.float32(dst), np.float32(src))
58     return fwd, inv
59
60
61 def _smooth_test_image(orig_w, orig_h, seed=506):
62     """Deterministic smooth float32 RGB image: smooth enough that both bilinear
63     implementations agree on identical sample positions, textured enough that a
64     sub-pixel sampling shift shows up loudly."""
65     import cv2
66
67     rng = np.random.default_rng(seed)
68     img = rng.random((orig_h, orig_w, 3)).astype(np.float32)
69     return cv2.GaussianBlur(img, (0, 0), sigmaX=2.0)
70
71
72 # ----- #
73 # The resize-fidelity lock (CPU-runnable: torch + cv2, no GPU / no WASB repo)
74 # ----- #
75 def test_affine_grid_matches_cv2_warpaffine():
76     """grid_sample over build_affine_grid == cv2.warpAffine on WASB's production
77     geometry (16:9 frame, the badminton 512x288-shaped target)."""
78     torch = pytest.importorskip("torch")
79     cv2 = pytest.importorskip("cv2")
80     import torch.nn.functional as F
81
82     orig_w, orig_h, dst_w, dst_h = 192, 108, 64, 36 # 16:9 ? 16:9, scale 3.0
83     img = _smooth_test_image(orig_w, orig_h)
84     fwd, inv = _wasb_affine_matrices(orig_w, orig_h, dst_w, dst_h)
85
86     ref = cv2.warpAffine(img, fwd, (dst_w, dst_h), flags=cv2.INTER_LINEAR)
87
88     grid = wi.build_affine_grid(inv, dst_w, dst_h, orig_w, orig_h)
89     assert tuple(grid.shape) == (1, dst_h, dst_w, 2)
90     chw = torch.from_numpy(img).permute(2, 0, 1).unsqueeze(0)
91     got = (
92         F.grid_sample(
93             chw, grid, mode="bilinear", align_corners=False, padding_mode="border"

```

```

94         )
95         .squeeze(0)
96         .permute(1, 2, 0)
97         .numpy()
98     )
99
100     diff = np.abs(got - ref)
101     # cv2 quantizes bilinear coefficients (INTER_BITS); anything beyond that noise
102     # floor means the geometry diverged. The naive-resize failure below is ~100x this.
103     assert diff.mean() < 1e-3, f"mean |?| {diff.mean():.2e} vs cv2.warpAffine"
104     assert diff.max() < 0.02, f"max |?| {diff.max():.2e} vs cv2.warpAffine"
105
106
107     def test_naive_interpolate_does_not_match_cv2_warppaffine():
108         """The #506 regression signature: F.interpolate is NOT the affine warp. If this
109         test ever fails (interpolate suddenly 'matches'), the reference setup is broken ?
110         it exists so the grid can never be quietly replaced by a plain resize."""
111         torch = pytest.importorskip("torch")
112         cv2 = pytest.importorskip("cv2")
113         import torch.nn.functional as F
114
115         orig_w, orig_h, dst_w, dst_h = 192, 108, 64, 36
116         img = _smooth_test_image(orig_w, orig_h)
117         fwd, inv = _wasb_affine_matrices(orig_w, orig_h, dst_w, dst_h)
118
119         ref = cv2.warpAffine(img, fwd, (dst_w, dst_h), flags=cv2.INTER_LINEAR)
120         chw = torch.from_numpy(img).permute(2, 0, 1).unsqueeze(0)
121
122         grid = wi.build_affine_grid(inv, dst_w, dst_h, orig_w, orig_h)
123         affine = (
124             F.grid_sample(
125                 chw, grid, mode="bilinear", align_corners=False, padding_mode="border"
126             )
127             .squeeze(0)
128             .permute(1, 2, 0)
129             .numpy()
130         )
131         naive = (
132             F.interpolate(chw, size=(dst_h, dst_w), mode="bilinear", align_corners=False)
133             .squeeze(0)
134             .permute(1, 2, 0)
135             .numpy()
136         )
137
138         affine_err = np.abs(affine - ref).mean()
139         naive_err = np.abs(naive - ref).mean()
140         # The half-pixel-convention shift is a real, systematic error ? far above the
141         # affine grid's coefficient-quantization noise floor.
142         assert naive_err > 1e-3, f"naive resize unexpectedly matches cv2 ({naive_err:.2e})"
143         assert naive_err > 5 * affine_err, (
144             f"affine grid ({affine_err:.2e}) must beat naive resize ({naive_err:.2e}) "
145             f"decisively ? otherwise the #506 lesson is unlearned"
146         )
147
148
149     # ----- #
150     # Cache keying: nvdec detections are trajectory-level (not byte-level) equivalents
151     # of cpu ones, so the resumable cache must never be shared across backends.
152     # ----- #
153     def _manifest(**over):
154         m = {
155             "version": wi.CACHE_VERSION,
156             "frames_dir": "video:/x.mp4",
157             "weights": "/w.pth.tar",
158             "sport": "badminton",

```

```

159         "frames_in": 3,
160         "frames_total": 100,
161         "device": "cuda",
162     }
163     m.update(over)
164     return m
165
166
167 def test_cache_never_shared_across_decode_backends():
168     kw = dict(
169         frames_dir="video:/x.mp4",
170         weights="/w.pth.tar",
171         sport="badminton",
172         frames_in=3,
173         frames_total=100,
174         device="cuda",
175     )
176     # Pre-#506 manifests carry no decode_backend field == cpu-decoded: still valid
177     # for a cpu run (no invalidation on upgrade), NEVER for an nvdec run.
178     legacy = _manifest()
179     assert wi.manifest_compatible(legacy, **kw)
180     assert not wi.manifest_compatible(legacy, **kw, decode_backend="nvdec_resident")
181     # And an nvdec cache must not be replayed by a cpu run.
182     nv = _manifest(decode_backend="nvdec_resident")
183     assert wi.manifest_compatible(nv, **kw, decode_backend="nvdec_resident")
184     assert not wi.manifest_compatible(nv, **kw)
185
186
187 # ----- #
188 # CLI guards: the nvdec branch must fail loud BEFORE any torch/GPU work.
189 # ----- #
190 def test_cli_nvdec_resident_requires_video(monkeypatch, tmp_path):
191     monkeypatch.setattr(
192         sys,
193         "argv",
194         [
195             "wasb_infer.py",
196             "--decode-backend",
197             "nvdec_resident",
198             "--frames_dir",
199             str(tmp_path),
200             "--weights",
201             "w.pth.tar",
202             "--out",
203             str(tmp_path / "o.csv"),
204         ],
205     )
206     with pytest.raises(SystemExit, match="requires --video"):
207         wi.main()
208
209
210 def test_cli_nvdec_resident_requires_cuda_device(monkeypatch, tmp_path):
211     monkeypatch.setattr(
212         sys,
213         "argv",
214         [
215             "wasb_infer.py",
216             "--decode-backend",
217             "nvdec_resident",
218             "--video",
219             "v.mp4",
220             "--device",
221             "cpu",
222             "--weights",
223             "w.pth.tar",

```

```

224         "--out",
225         str(tmp_path / "o.csv"),
226     ],
227 )
228 with pytest.raises(SystemExit, match="cuda"):
229     wi.main()
230
231
232 def test_cli_rejects_unknown_decode_backend(monkeypatch, tmp_path, capsys):
233     monkeypatch.setattr(
234         sys,
235         "argv",
236         [
237             "wasb_infer.py",
238             "--decode-backend",
239             "gpu", # not a backend; argparse choices reject it
240             "--video",
241             "v.mp4",
242             "--weights",
243             "w.pth.tar",
244             "--out",
245             str(tmp_path / "o.csv"),
246         ],
247     )
248     with pytest.raises(SystemExit):
249         wi.main()
250     assert "invalid choice" in capsys.readouterr().err
251
252
253 # ----- #
254 # The #506 unit gate: nvdec_resident trajectory ? cpu trajectory on a real clip.
255 # HARDWARE test ? skips unless the box provides everything. Run on the GPU box
256 # (inside the torch-2.x wasb env, which has torch+cuda, PyNvVideoCodec, cv2):
257 #
258 # WASB_REPO_DIR=~/.models/WASB-SBDT \
259 # WASB_WEIGHTS=~/.models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
260 # WASB_PARITY_VIDEO=/path/to/short_fixture_clip.mp4 \
261 # python -m pytest tests/test_wasb_nvdec_resident.py -k parity -v
262 # ----- #
263 def _load_traj(path):
264     out = {}
265     with open(path, newline="") as f:
266         for row in csv.DictReader(f):
267             out[int(float(row["Frame"]))] = (
268                 int(float(row["Visibility"])),
269                 float(row["X"]),
270                 float(row["Y"]),
271             )
272     return out
273
274
275 def test_nvdec_resident_trajectory_matches_cpu_on_fixture(tmp_path):
276     repo = osp.expanduser(osp.environ.get("WASB_REPO_DIR", ""))
277     weights = osp.expanduser(osp.environ.get("WASB_WEIGHTS", ""))
278     video = osp.expanduser(osp.environ.get("WASB_PARITY_VIDEO", ""))
279     if not (repo and weights and video):
280         pytest.skip("set WASB_REPO_DIR + WASB_WEIGHTS + WASB_PARITY_VIDEO to run")
281     for p in (osp.join(repo, "src"), weights, video):
282         if not osp.exists(p):
283             pytest.skip(f"missing {p}")
284     torch = pytest.importorskip("torch")
285     if not torch.cuda.is_available():
286         pytest.skip("needs a CUDA GPU")
287     pytest.importorskip("PyNvVideoCodec")
288

```

```

289 # Stage wasb_infer.py into the WASB repo src/ (exactly as the runners do) and import
290 # THAT copy, so its WASB imports and Hydra's configs/ dir resolve.
291 import shutil
292
293 from backend.pipeline.detectors import native_wasb_runner as nwr
294
295 src = osp.join(repo, "src")
296 staged = osp.join(src, "wasb_infer.py")
297 shutil.copy(nwr._INFER_PY, staged)
298 sys.path.insert(0, src)
299 try:
300     spec = importlib.util.spec_from_file_location("wasb_infer_staged", staged)
301     mod = importlib.util.module_from_spec(spec)
302     assert spec.loader is not None
303     spec.loader.exec_module(mod)
304
305     cpu_csv = str(tmp_path / "cpu_traj.csv")
306     gpu_csv = str(tmp_path / "gpu_traj.csv")
307     mod.run_video_streaming(video, weights, cpu_csv, fresh=True)
308     mod.run_video_nvdec_resident(video, weights, gpu_csv, fresh=True)
309
310     cpu, gpu = _load_traj(cpu_csv), _load_traj(gpu_csv)
311     common = sorted(set(cpu) & set(gpu))
312     assert len(common) >= 10, "fixture too short to be a meaningful gate"
313     agree = sum(1 for f in common if cpu[f][0] == gpu[f][0]) / len(common)
314     dists = [
315         float(np.hypot(cpu[f][1] - gpu[f][1], cpu[f][2] - gpu[f][2]))
316         for f in common
317         if cpu[f][0] == 1 and gpu[f][0] == 1
318     ]
319     # Validated on the L4 (#506): 96.3% visibility agreement, median 0.036 px.
320     assert agree >= 0.95, f"visibility agreement {agree:.3f} < 0.95"
321     assert dists, "no frames visible in both trajectories"
322     assert float(np.median(dists)) <= 1.0, (
323         f"median both-visible distance {float(np.median(dists)):.3f}px > 1.0px"
324     )
325 finally:
326     sys.path.remove(src)
327

```

4. user (2026-07-07T04:03:23.457Z)

```

876 diff --git a/docs/CLOUD_GPU_DRYRUN_GUIDE.md b/docs/CLOUD_GPU_DRYRUN_GUIDE.md
877 index 8e263b4..6021064 100644
878 --- a/docs/CLOUD_GPU_DRYRUN_GUIDE.md
879 +++ b/docs/CLOUD_GPU_DRYRUN_GUIDE.md
880 @@ -93,8 +93,9 @@ Watch it finish (look for `SETUP_DONE` and `torch ... cuda_available
True`):
881     gcloud compute ssh rally-gpu --zone="$ZONE" --command='tail -n 20 ~/setup.log'
882     ```
883
884     -> ? You're ready when the log shows the WASB env built with `torch 1.11.0+cu113 |
cuda_available True
885     -> | device NVIDIA L4` and `SETUP_DONE`. (The 5.8 MB detector weights are now staged.)
886     +> ? You're ready when the log shows the WASB env built with `torch 2.6.0+cu124 |
cuda_available True
887     +> | device NVIDIA L4` (plus a `PyNvVideoCodec ... OK` line ? the #506 GPU-resident
decode dep) and
888     +> `SETUP_DONE`. (The 5.8 MB detector weights are now staged.)
889
890     ---
891
892 diff --git a/docs/PACKAGING_PLAN.md b/docs/PACKAGING_PLAN.md
893 index 9822606..5a2873a 100644
894 --- a/docs/PACKAGING_PLAN.md

```

```

895     +++ b/docs/PACKAGING_PLAN.md
896     @@ -20,7 +20,7 @@
897     ---
898
899     ## 0. TL;DR
900     -Ship the engine as ONE pip-installable artifact whose behaviour is selected entirely
    by the already-merged decoupled-compute seam** (`deployment.profile` / `compute_target` /
    Storage+Compute backends), fronted by a friendly launcher + a Windows installer ? not a
    frozen monolith, not an Electron/Tauri shell. The hard constraint that forces scope: torch
    is deliberately out of `pyproject` (needs `--index-url`) and native WASB needs a second,
    incompatible* torch env (conda py3.8 + torch 1.11+cu113) shelled out via `$WASB_PYTHON`
    `[verified native_wasb_runner.py:89]` ? so no single venv/freeze can deliver the GPU-quality
    path. The honest v1 is a LIGHT tier (Gemini + motion + cpu-mock, zero* torch/GPU) that
    preserves CUJ-1.5/8; the native-WASB GPU path (CUJ-6/7) ships owner-only until two blockers
    clear (WASB-C3 weight provenance + a reproducible truly-local config). LIGHT is the FIRST
    shippable increment, not the terminal scope ? the sole completion goal (north star, L10) is the
    FULL quality path: native WASB + BYO training + in-app annotation. Delivery is cross-
    platform (Windows/macOS/Linux): the same local webserver powers the UI on all three, with NO
    native component** (L9). Five small engine prerequisites must land before any packaging push
    ($7 P0). The single most actionable finding: all mutable state is CWD-relative today and
    `--setup`/server disagree on `config.json` ? a silent data-loss footgun that P0 must fix
    first.
901     +Ship the engine as ONE pip-installable artifact whose behaviour is selected entirely
    by the already-merged decoupled-compute seam** (`deployment.profile` / `compute_target` /
    Storage+Compute backends), fronted by a friendly launcher + a Windows installer ? not a
    frozen monolith, not an Electron/Tauri shell. The hard constraint that forces scope: torch
    is deliberately out of `pyproject` (needs `--index-url`) and native WASB needs a second,
    isolated* torch env (conda py3.10 + torch 2.6+cu124 + PyNvVideoCodec since #506) shelled out via
    `$WASB_PYTHON` `[verified native_wasb_runner.py:89]` ? so no single venv/freeze can deliver the
    GPU-quality path. The honest v1 is a LIGHT tier (Gemini + motion + cpu-mock, zero*
    torch/GPU) that preserves CUJ-1.5/8; the native-WASB GPU path (CUJ-6/7) ships owner-only
    until two blockers clear (WASB-C3 weight provenance + a reproducible truly-local config).
    LIGHT is the FIRST shippable increment, not the terminal scope ? the sole completion goal
    (north star, L10) is the FULL quality path: native WASB + BYO training + in-app annotation. Delivery is cross-platform (Windows/macOS/Linux): the same local webserver powers the UI on
    all three, with NO native component** (L9). Five small engine prerequisites must land before
    any packaging push** ($7 P0). The single most actionable finding: all mutable state is CWD-
    relative today and `--setup`/server disagree on `config.json` ? a silent data-loss footgun
    that P0 must fix first.
902
903     ## 1. Problem & goal
904     Goal: a non-technical user (the primary persona: an India coach/parent on Windows ?
    `docs/UI_PROPOSAL.md` §2) downloads and runs the indexer without sourcing the repo. The
    app spins up a local web service for (a) inference video?rallies?reel, (b) BYO training,
    and (c) rally annotation if needed ? runnable locally and against cloud services ? and
    preserves every completed CUJ ($ CUJ map).
905     @@ -75,7 +75,7 @@ Build `kheisutra/web` (`npm ci && npm run build`) at package time
    and vendor
906     | Tier | Audience | Contents | Install UX |
907     |-----|-----|-----|-----|
908     | LIGHT (v1 first increment) | Non-tech India coach/parent (Win/Mac/Linux); any CPU
    box | wheel + bundled SPA + Gemini/motion/cpu-mock; no torch/GPU/weights; ffmpeg via per-OS
    offer; defaults truly-local; user-supplied Gemini key | Cross-platform, no native component:
    `uv tool install` / a generated install script + a pure-Python launcher (`indexer --app` ?
    boot server ? `webbrowser.open 127.0.0.1:8000`, free-port fallback). Optional thin per-OS
    shortcut (`bat`/`.command`/`.desktop`) wrapping the same command. First run: cpu-mock self-test
    ? 3-step wizard. |
909     -| HEAVY / local-GPU (owner + opt-in power user) | Owner w/ golden corpus + CUDA box
    | LIGHT + micromamba WASB env (py3.8/torch1.11) via `$WASB_PYTHON`; main torch out-of-band via
    uv; weights SHA-256-pinned. Gated on C3 + reproducible config; ONNX supersedes this env |
    opt-in "Enable local GPU" step; resumable progress (multi-GB on Indian bandwidth); post-install
    `torch.cuda.is_available()` validate; falls back to LIGHT. |
910     +| HEAVY / local-GPU (owner + opt-in power user) | Owner w/ golden corpus + CUDA box
    | LIGHT + micromamba WASB env (py3.10/torch2.6, #506) via `$WASB_PYTHON`; main torch out-of-band
    via uv; weights SHA-256-pinned. Gated on C3 + reproducible config; ONNX supersedes this env

```

```

| opt-in "Enable local GPU" step; resumable progress (multi-GB on Indian bandwidth); post-
install `torch.cuda.is_available()` validate; falls back to LIGHT. |
911 | **CLOUD / server** | GCP deploy + BYO-GPU-VM | the `deploy/cloudrun/Dockerfile` image
(micromamba-ified); cloud-serving + gcs; behind Cloudflare Access | `gcloud run jobs create` per
`deploy/cloudrun/README.md`, or `deploy/gcp` VM scripts. Bind? `0.0.0.0` (config-gated) + edge-
auth mandatory. |
912
913 ## 5. Threat model / risk table (local appliance)
914 diff --git a/tests/test_compute_backend.py b/tests/test_compute_backend.py
915 index a41ce7d..a49c03a 100644
916 --- a/tests/test_compute_backend.py
917 +++ b/tests/test_compute_backend.py
918 @@ -690,7 +690,14 @@ def test_dockerfile_has_required_directives():
919     assert "FROM nvidia/cuda" in text # GPU base (CUDA)
920     assert "ffmpeg" in text # stitch/decode
921     assert "fonts-dejavu" in text # the watermark filter font
922     - assert "torch==1.11.0+cu113" in text # the verified WASB stack
923     + # The #506 modernized WASB stack (L4-validated): torch 2.6 cu124 + PyNvVideoCodec,
and
924     + # the NVDEC/NVENC driver userspace libs PyNvVideoCodec dlopens (compute-only GPU
hosts
925     + # inject neither) ? a silent drop of any of these bricks
decode_backend=nvdec_resident.
926     + assert "torch==2.6.0" in text
927     + assert "download.pytorch.org/whl/cu124" in text
928     + assert "PyNvVideoCodec" in text
929     + assert "libnvcuvid.so.1" in text and "libnvidia-encode.so.1" in text
930     + assert "torch==1.11.0+cu113" not in text # the old stack is gone, not co-installed
931     assert "WASB-SBDT" in text # the shuttle detector repo
932     assert 'ENTRYPOINT ["python3", "-m", "backend.worker"]' in text
933     assert "WASB_PYTHON=/opt/conda/envs/wasb/bin/python" in text
934 diff --git a/tests/test_config_wasb.py b/tests/test_config_wasb.py
935 index 9d304df..887b9c2 100644
936 --- a/tests/test_config_wasb.py
937 +++ b/tests/test_config_wasb.py
938 @@ -102,6 +102,68 @@ def test_wasb_gpu_feed_knobs_reject_invalid_values():
939     AppConfig(indexing={"wasb": {"batch_size": -8}})
940
941
942 +def test_wasb_decode_backend_default_off_and_reaches_native_runner():
943     + """#506: decode_backend defaults to "cpu" (the flag is default-OFF until the full
944     + 15-clip golden gate at the SERVED preset passes) and an explicit override reaches
945     + the native runner's config."""
946     + from backend.config.models import WasbIndexConfig
947     + from backend.pipeline.detectors.native_wasb_runner import NativeWasbConfig
948     +
949     + assert WasbIndexConfig().decode_backend == "cpu"
950     + assert (
951     +     NativeWasbConfig.from_indexing_cfg(AppConfig().indexing).decode_backend
952     +     == "cpu"
953     + )
954     + idx = AppConfig(indexing={"wasb": {"decode_backend": "nvdec_resident"}}).indexing
955     + assert NativeWasbConfig.from_indexing_cfg(idx).decode_backend == "nvdec_resident"
956
957
958 +def test_wasb_decode_backend_env_override_beats_config(monkeypatch):
959     + from backend.pipeline.detectors.native_wasb_runner import NativeWasbConfig
960     +
961     + monkeypatch.setenv("WASB_DECODE_BACKEND", "nvdec_resident")
962     + cfg = NativeWasbConfig.from_indexing_cfg(AppConfig().indexing)
963     + assert cfg.decode_backend == "nvdec_resident"
964
965
966 +def test_wasb_decode_backend_rejects_typos_and_non_cuda_device():
967     + """A typo'd backend and the contradictory nvdec+cpu-device combo must fail at

```



```

968 + config load, not deep inside a paid GPU run. device=auto stays allowed (the
969 + native runner's healthcheck hard-fails if auto resolves to cpu)."""
970 + import pytest
971 +
972 + from backend.config.models import WasbIndexConfig
973 +
974 + with pytest.raises(ValueError):
975 +     WasbIndexConfig(decode_backend="nvdec") # Literal rejects unknown backends
976 + with pytest.raises(ValueError, match="requires device"):
977 +     WasbIndexConfig(decode_backend="nvdec_resident", device="cpu")
978 + with pytest.raises(ValueError, match="requires device"):
979 +     AppConfig(
980 +         indexing={
981 +             "wasb": {"decode_backend": "nvdec_resident", "device": "mps"}
982 +         }
983 +     )
984 + # cuda (the default) and auto are both valid hosts for the GPU-resident path.
985 + assert (
986 +     WasbIndexConfig(decode_backend="nvdec_resident").decode_backend
987 +     == "nvdec_resident"
988 + )
989 + assert (
990 +     WasbIndexConfig(decode_backend="nvdec_resident", device="auto").device
991 +     == "auto"
992 + )
993 +
994 +
995 +def test_cloud_serving_preset_keeps_decode_backend_default_off():
996 +    """The rollout gate (#506): the cloud-serving preset must NOT opt into
997 +    nvdec_resident until the full golden set passes at the SERVED preset ? a
998 +    regression here would silently flip the default in production."""
999 +    from backend.config.presets import preset_for
1000 +
1001 +    assert "decode_backend" not in preset_for("cloud-serving")["indexing"]["wasb"]
1002 +
1003 +
1004 +def test_live_indexing_knobs_declared():
1005 +    fields = IndexingConfig.model_fields
1006 +    assert "yolo_prefetch_workers" in fields
1007 +diff --git a/tests/test_native_wasb_runner.py b/tests/test_native_wasb_runner.py
1008 +index b636300..9ae9208 100644
1009 +--- a/tests/test_native_wasb_runner.py
1010 ++++ b/tests/test_native_wasb_runner.py
1011 +@@ -458,6 +458,80 @@ def test_csv_name_matches_wsl_contract(tmp_path):
1012 +     assert os.path.basename(out) == f"{key}_wasb.csv"
1013 +
1014 +
1015 +## --- #506: GPU-resident NVDEC decode backend
1016 +-----
1017 +def test_run_predict_omits_decode_backend_flag_by_default(tmp_path):
1018 +    """decode_backend defaults to "cpu" ? the flag is NOT passed, so the default argv
1019 +    (and its byte-parity guarantee) is unchanged by #506."""
1020 +    _, _out, _rm, argv, _cwd, _key = _drive_success(
1021 +        NativeWasbConfig(weights_path="/m/w.pth"), tmp_path
1022 +    )
1023 +    assert "--decode-backend" not in argv
1024 +
1025 +def test_run_predict_nvdec_resident_threads_flag_and_skips_cv2_paths(tmp_path):
1026 +    """nvdec_resident decodes on the GPU inside wasb_infer.py: the flag is threaded and
1027 +    NEITHER cv2 path flag (--stream-video / --frames_out_dir) rides along; there is no
1028 +    frame cache to clean."""
1029 +    _, out, rm, argv, _cwd, _key = _drive_success(
1030 +        NativeWasbConfig(weights_path="/m/w.pth", decode_backend="nvdec_resident"),
1031 +        tmp_path,

```

```

1032 + )
1033 + assert out is not None
1034 + assert argv[argv.index("--decode-backend") + 1] == "nvdec_resident"
1035 + assert "--stream-video" not in argv and "--frames_out_dir" not in argv
1036 + assert argv[argv.index("--device") + 1] == "cuda"
1037 + rm.assert_not_called()
1038 +
1039 +
1040 +def
test_run_predict_nvdec_resident_auto_without_gpu_aborts_before_subprocess(tmp_path):
1041 +     """device=auto resolving to cpu cannot host NVDEC decode: run_predict fails BEFORE
1042 +     launching the (paid) inference subprocess ? only the CUDA probe runs."""
1043 +     r = NativeWasbRunner(
1044 +         NativeWasbConfig(
1045 +             weights_path="/m/w.pth", device="auto", decode_backend="nvdec_resident"
1046 +         )
1047 +     )
1048 +     with patch.object(r, "_copy_infer_script", return_value=True), patch.object(
1049 +         r, "_run", return_value=_proc(0, stdout="torch 2.6.0 cuda False")
1050 +     ) as spy:
1051 +         out = r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
1052 +         assert out is None
1053 +         assert spy.call_count == 1 # the probe only; wasb_infer.py never launched
1054 +
1055 +
1056 +def test_healthcheck_rejects_unknown_decode_backend(tmp_path):
1057 +     """A typo'd decode_backend fails fast with a clear message, before any
subprocess."""
1058 +     r = NativeWasbRunner(_ok_cfg(tmp_path, decode_backend="nvdec"))
1059 +     with patch.object(r, "_run") as spy:
1060 +         ok, msg = r.healthcheck()
1061 +         assert not ok and "decode_backend" in msg
1062 +         spy.assert_not_called()
1063 +
1064 +
1065 +def test_healthcheck_nvdec_resident_requires_effective_cuda(tmp_path):
1066 +     """auto?cpu fallback is fine for the cpu decode path but NOT for nvdec_resident:
1067 +     the healthcheck must hard-fail instead of warn-and-proceed."""
1068 +     r = NativeWasbRunner(
1069 +         _ok_cfg(tmp_path, device="auto", decode_backend="nvdec_resident")
1070 +     )
1071 +     with patch.object(
1072 +         r, "_run", return_value=_proc(0, stdout="torch 2.6.0 cuda False")
1073 +     ):
1074 +         ok, msg = r.healthcheck()
1075 +         assert not ok and "nvdec_resident" in msg
1076 +
1077 +
1078 +def test_healthcheck_nvdec_resident_ok_with_cuda(tmp_path):
1079 +     r = NativeWasbRunner(
1080 +         _ok_cfg(tmp_path, device="cuda", decode_backend="nvdec_resident")
1081 +     )
1082 +     with patch.object(
1083 +         r, "_run", return_value=_proc(0, stdout="torch 2.6.0 cuda True")
1084 +     ):
1085 +         ok, _msg = r.healthcheck()
1086 +         assert ok
1087 +
1088 +
1089 + # --- the detector_impl='native' seam (parallels the stub seam)
-----
1090 + def test_resolve_runner_native_selects_native_runner_for_wasb(monkeypatch):
1091 +     monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
1092 +

```

5. assistant (2026-07-07T04:03:33.208Z)

Now let me read the actual repo files to verify test validity claims.

6. user (2026-07-07T04:03:34.672Z)

```
1      """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3      WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4      own building blocks (ImageDataset transform + detector + online tracker) to run on
5      any folder of extracted frames and emit a trajectory CSV in the exact shape the
6      indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8          Frame,Visibility,X,Y
9
10     It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11     imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12     (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14     Resumable across reboots
15     -----
16     The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17     the slow, expensive part; the CPU tracker pass that turns detections into a
18     trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19     resume mid-stream). We exploit that split:
20
21     * The detector pass writes its per-window raw outputs incrementally to
22       ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23       records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24       most ``--flush-every`` windows of GPU work instead of the whole pass.
25     * On restart we replay the cached windows, skip the DataLoader ahead to the first
26       un-cached window, and continue. Once every window is cached, the detector pass
27       is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28       the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29       not the GPU hour.
30
31     The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32     to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33     weights, sport, window size, frame count, device, or decode backend change.
34
35     GPU-resident decode (#506)
36     -----
37     ``--decode-backend nvdec_resident`` (default ``cpu``) moves decode + resize + normalize
38     onto the GPU with no host round-trip: PyNvVideoCodec NVDEC decode ? affine
39     ``grid_sample`` resize replicating WASB's ``cv2.warpAffine`` exactly ? GPU ImageNet
40     normalize ? detector. Validated on a real L4 (issue #506): ~1.7x the cv2 streaming
41     path with rally F1 preserved on the hard golden clip. CUDA-only; needs the torch-2.x +
42     PyNvVideoCodec env (the WSL torch-1.11 env keeps ``cpu``, its behaviour untouched).
43
44     Usage (in WSL, from ~/models/WASB-SBDT/src):
45     python wasb_infer.py --frames_dir /path/frames --weights
46     ../pretrained_weights/wasb_badminton_best.pth.tar \
47     --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
48     every 1000] [--fresh]
49     """
50
51     import argparse
52     import csv
53     import glob
54     import json
55     import logging
56     import os
57     import os.path as osp
58     from collections import defaultdict
59     from contextlib import contextmanager
```

```

58
59     logger = logging.getLogger(__name__)
60
61     CACHE_VERSION = 1
62     _DET_FILE = "det_raw.jsonl"
63     _MANIFEST_FILE = "manifest.json"
64
65     # Pixel-decode backends (#506). "cpu" = the historical cv2 paths (disk frames or
66     # --stream-video); "nvdec_resident" = GPU-resident NVDEC decode + affine grid_sample
67     # resize + normalize (torch 2.x + PyNvVideoCodec, CUDA-only).
68     DECODE_BACKENDS = ("cpu", "nvdec_resident")
69
70     # ImageNet statistics ? MUST match WASB's build_img_transforms Normalize (the GPU-
71     resident
72     # frontend normalizes on-device instead of through torchvision's transform).
73     IMAGENET_MEAN = (0.485, 0.456, 0.406)
74     IMAGENET_STD = (0.229, 0.224, 0.225)
75
76     def _build_cfg(weights: str, sport: str, device: str = "cuda"):
77         """Compose the WASB eval config via Hydra, overriding model/weights/device.
78
79         ``device`` is ``cuda`` (default ? the historical WSL behaviour, byte-parity
80         preserved),
81         ``mps`` (Apple) or ``cpu``. Only the single-GPU CUDA path requests
82         ``runner.gpus=[0]``;
83         cpu/mps run with no GPU list so the detector places tensors on ``device``.
84         """
85         from hydra import compose, initialize
86
87         gpus = "[0]" if device == "cuda" else "[]" # single GPU on cuda; none for cpu/mps
88         with initialize(config_path="configs", version_base=None):
89             cfg = compose(
90                 config_name="eval",
91                 overrides=[
92                     f"dataset={sport}",
93                     "model=wasb",
94                     f"detector.model_path={weights}",
95                     f"runner.device={device}",
96                     f"runner.gpus={gpus}", # default config assumes multiple GPUs; pin to
97                     this box
98                 ],
99             )
100             return cfg
101
102     def _frame_id(path: str) -> int:
103         """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
104         180)."""
105         stem = osp.splitext(osp.basename(path))[0]
106         digits = "".join(ch for ch in stem if ch.isdigit())
107         return int(digits) if digits else -1
108
109     # ----- #
110     # Runtime-compat shims for the torch-2.x / NumPy-2.x env (#506). Both are exact
111     # no-ops on the verified WSL torch-1.11 + numpy<1.24 stack.
112     # ----- #
113     def _ensure_np_inf() -> None:
114         """NumPy 2.0 removed the ``np.Inf`` alias, which WASB's OnlineTracker still reads.
115         Restore it before any tracker use; a no-op on NumPy 1.x."""
116         import numpy as np
117
118         if not hasattr(np, "Inf"):
119             np.Inf = np.inf # type: ignore[attr-defined]

```

```
118
119
120     @contextmanager
```

7. user (2026-07-07T04:03:36.420Z)

```
#!/usr/bin/env bash
#
```

Native (no-WSL) WASB shuttle-detector setup for a Linux GPU box ? the M3.5 enabler for real trajectory extraction. Replicates docs/TRACKNET_WSL_SETUP.md's WSL conda env *natively* on Linux. Run AFTER ./bootstrap.sh --gpu (which provides CUDA torch for the main venv); this builds a SEPARATE conda env for WASB ? never co-installed with the indexer's torch (framework conflict).

```
#
#506 modernized stack (validated end-to-end on a real L4, 2026-07-06): py3.10 +
torch 2.6.0+cu124 (WASB HRNet ports clean; cpu F1 ? the torch-1.11 baseline) +
PyNvVideoCodec for the GPU-resident decode backend (indexing.wasb.decode_backend.py:
nvdec_resident, ~1.7x). Step [2/6] stages the NVDEC/NVENC driver userspace libs
PyNvVideoCodec needs (compute-only driver images ship neither).
```

```
#
Requires an NVIDIA GPU + driver (the DO `gpu-*-base` image ships it).
```

```
#
? WASB-C3: the pretrained badminton weights' license / source-dataset provenance is
verified here. Fine for internal extraction + Gen-0 bootstrapping; clear it before sharing
any derived trajectories or model. See docs/DECOUPLED_COMPUTE/06.
```

```
set -euo pipefail
```

```
MODELS_DIR="${RALLY_MODELS_DIR:-$HOME/models}"
CONDA_DIR="${CONDA_DIR:-$HOME/miniconda3}"
WASB_REPO="$MODELS_DIR/WASB-SBDT"
mkdir -p "$MODELS_DIR"
```

```
echo "[gpu] [1/6] miniconda (native Linux)"
if [ ! -x "$CONDA_DIR/bin/conda" ]; then
    curl -sSL https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -o /tmp/mc.sh
    bash /tmp/mc.sh -b -p "$CONDA_DIR" && rm -f /tmp/mc.sh
fi
```

shellcheck disable=SC1091

```
source "$CONDA_DIR/etc/profile.d/conda.sh"
```

```
echo "[gpu] [2/6] NVDEC/NVENC driver userspace libs (#506: PyNvVideoCodec needs BOTH)"
```

Compute-only driver images (GCE DLVM, DO gpu-*-base) ship neither libnvcuvid.so.1 nor libnvidia-encode.so.1. Extract them from the .run matching the RUNNING driver so the userspace/kernel versions can never skew (see the #506 / gpu-vm-setup-gotchas note).

```
if [ ! -f /usr/lib/x86_64-linux-gnu/libnvcuvid.so.1 ] || [ ! -f /usr/lib/x86_64-linux-
gnu/libnvidia-encode.so.1 ]; then
    DRV="$(nvidia-smi --query-gpu=driver_version --format=csv,noheader | head -1)"
```

8. user (2026-07-07T04:03:45.781Z)

```
139:         kwargs.setdefault("weights_only", False)
```

```

1233: def main():
1235:     ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
1236:     ap.add_argument(
1239:     ap.add_argument(
1242:     ap.add_argument(
1248:     ap.add_argument(
1258:     ap.add_argument("--weights", required=True)
1259:     ap.add_argument("--out", required=True)
1260:     ap.add_argument("--sport", default="badminton")
1261:     ap.add_argument(
1267:     ap.add_argument(
1270:     ap.add_argument("--batch-size", type=int, default=8)
1271:     ap.add_argument("--num-workers", type=int, default=4)
1272:     ap.add_argument(
1279:     ap.add_argument(
1282:     ap.add_argument(
1287:     ap.add_argument(
1293:     ap.add_argument(
1298:     if args.decode_backend == "nvdec_resident":
1301:         if not args.video:
1303:             if args.device != "cuda":
1309:                 args.video,
1310:                 args.weights,
1311:                 args.out,
1312:                 args.sport,
1313:                 args.limit,
1314:                 batch_size=args.batch_size,
1315:                 log_every_batches=args.log_every_batches,
1316:                 cache_dir=args.cache_dir,
1317:                 flush_every=args.flush_every,
1318:                 fresh=args.fresh,
1322:     if args.stream_video:
1323:         if not args.video:
1326:             args.video,
1327:             args.weights,
1328:             args.out,
1329:             args.sport,
1330:             args.limit,
1331:             batch_size=args.batch_size,
1332:             log_every_batches=args.log_every_batches,
1333:             cache_dir=args.cache_dir,
1334:             flush_every=args.flush_every,
1335:             fresh=args.fresh,
1336:             device=args.device,
1337:             prefetch_batches=args.prefetch_batches,
1341:     frames_dir = args.frames_dir
1342:     if args.video:
1344:         args.video, args.frames_out_dir or (args.video + "_frames")
1350:         args.weights,
1351:         args.out,
1352:         args.sport,
1353:         args.limit,
1354:         batch_size=args.batch_size,
1355:         num_workers=args.num_workers,
1356:         log_every_batches=args.log_every_batches,
1357:         cache_dir=args.cache_dir,
1358:         flush_every=args.flush_every,
1359:         fresh=args.fresh,
1360:         device=args.device,

```

9. user (2026-07-07T04:03:46.749Z)

```

516: def run(
517-     frames,
518-     weights: str,

```

```

519- out_csv: str,
520- sport: str = "badminton",
521- limit: int = 0,
522- batch_size: int = 8,
523- num_workers: int = 4,
524- log_every_batches: int = 50,
525- cache_dir: "str | None" = None,
526- flush_every: int = 1000,
527- fresh: bool = False,
528- frame_loader=None,
529- source_key: "str | None" = None,
530- device: str = "cuda",
531- prefetch_batches: int = 0,
532-) -> str:
533- """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
534-
535- Two equivalent ways to supply pixels (output is bit-identical between them):
536-
537- --
870: def run_video_streaming(
871-     video_path: str,
872-     weights: str,
873-     out_csv: str,
874-     sport: str = "badminton",
875-     limit: int = 0,
876-     batch_size: int = 8,
877-     log_every_batches: int = 50,
878-     cache_dir: "str | None" = None,
879-     flush_every: int = 1000,
880-     fresh: bool = False,
881-     device: str = "cuda",
882-     prefetch_batches: int = 0,
883-) -> str:
884- """Fast path: decode ``video_path`` on the fly and feed frames straight to the
885- detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
886- dominates a long clip). Output is bit-identical to the frames-on-disk path; see
887- ``run``'s docstring for why.
888- """
889- frame_loader, count_hint, release = make_video_frame_loader(video_path)
890- try:

```

10. user (2026-07-07T04:03:54.329Z)

```

1233 def main():
1234     ap = argparse.ArgumentParser()
1235     ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
1236     ap.add_argument(
1237         "--video", help="video file; frames are extracted internally (in WSL)"
1238     )
1239     ap.add_argument(
1240         "--frames_out_dir", help="where to extract frames when --video is used"
1241     )
1242     ap.add_argument(
1243         "--stream-video",
1244         action="store_true",
1245         help="FAST PATH: decode --video on the fly and feed frames straight to "
1246         "the detector (no PNG extraction). Output-equivalent to the disk path.",
1247     )
1248     ap.add_argument(
1249         "--decode-backend",
1250         default="cpu",
1251         choices=list(DECODE_BACKENDS),
1252         help="cpu (default) = the historical cv2 decode paths (disk frames or "
1253         "--stream-video). nvdec_resident = GPU-RESIDENT decode+resize+normalize "
1254         "(#506): PyNvVideoCodec NVDEC + cv2-faithful affine grid_sample, CUDA-only, "

```

```

1255         "~1.7x on an L4. Requires --video; detections cache separately from cpu "
1256         "(trajectory-level, not byte-level, equivalence).",
1257     )
1258     ap.add_argument("--weights", required=True)
1259     ap.add_argument("--out", required=True)
1260     ap.add_argument("--sport", default="badminton")
1261     ap.add_argument(
1262         "--device",
1263         default="cuda",
1264         choices=["cuda", "mps", "cpu"],
1265         help="torch device for the detector (default cuda = the WSL parity path)",
1266     )
1267     ap.add_argument(
1268         "--limit", type=int, default=0, help="cap #frames (for quick tests)"
1269     )
1270     ap.add_argument("--batch-size", type=int, default=8)
1271     ap.add_argument("--num-workers", type=int, default=4)
1272     ap.add_argument(
1273         "--prefetch-batches",
1274         type=int,
1275         default=0,
1276         help="streaming mode: overlap CPU batch assembly with GPU inference via a "
1277         "bounded producer thread of this queue depth (0 = off, the parity default)",
1278     )
1279     ap.add_argument(
1280         "--log-every-batches", type=int, default=50, help="progress log cadence"
1281     )
1282     ap.add_argument(
1283         "--cache-dir",
1284         default=None,
1285         help="resumable detector cache dir (default: <out>_wasbcache next to --out)",
1286     )
1287     ap.add_argument(
1288         "--flush-every",
1289         type=int,
1290         default=1000,
1291         help="fsync the detector cache every N windows (bounds reboot loss)",
1292     )
1293     ap.add_argument(
1294         "--fresh", action="store_true", help="ignore any existing cache and start over"
1295     )
1296     args = ap.parse_args()
1297
1298     if args.decode_backend == "nvdec_resident":
1299         # GPU-resident branch FIRST: it decodes the video itself (NVDEC), so neither the
1300         # --stream-video cv2 path nor the extract-frames fallback below may run.
1301         if not args.video:
1302             raise SystemExit("--decode-backend nvdec_resident requires --video")
1303         if args.device != "cuda":
1304             raise SystemExit(
1305                 "--decode-backend nvdec_resident requires --device cuda "
1306                 "(NVDEC decode runs on the GPU)"
1307             )
1308         run_video_nvdec_resident(
1309             args.video,
1310             args.weights,
1311             args.out,
1312             args.sport,

```

11. user (2026-07-07T04:03:56.038Z)

```

20: IMAGENET_MEAN = np.array([0.485, 0.456, 0.406], dtype=np.float32)
21: IMAGENET_STD = np.array([0.229, 0.224, 0.225], dtype=np.float32)
46: from utils.image import get_affine_transform
49: trans_input = get_affine_transform(c, s, 0, [INP_W, INP_H], inv=0)

```



```

50:     trans_out_inv = get_affine_transform(c, s, 0, [OUT_W, OUT_H], inv=1)
87: def normalize_chw_np(chw_rgb_float):
90:     out[c] = (chw_rgb_float[c] - IMAGENET_MEAN[c]) / IMAGENET_STD[c]
115:     return normalize_chw_np(chw)
125: class GpuFrontend:
142:     self.mean = torch.tensor(IMAGENET_MEAN, device=dev).view(3, 1, 1)
143:     self.std = torch.tensor(IMAGENET_STD, device=dev).view(3, 1, 1)
144:     # Affine sampling grid replicating WASB's cv2.warpAffine(trans_input, INTER_LINEAR)
146:     # inverse affine (trans_out_inv), normalized to [-1,1] with cv2's half-pixel
convention.
147:     ys, xs = torch.meshgrid(torch.arange(INP_H, dtype=torch.float32),
154:     self.grid = torch.stack([gx, gy], dim=-1).unsqueeze(0).to(dev) # [1,H,W,2]
171:     resized = F.grid_sample(chw, self.grid, mode="bilinear",

```

12. user (2026-07-07T04:04:02.857Z)

```

30         "MODEL": {"EXTRA": {"FINAL_CONV_KERNEL": 1, "PRETRAINED_LAYERS": ["*"],
31             "STEM": {"INPLANES": 64, "STRIDES": [1, 1]},
32             "STAGE1": {"NUM_MODULES": 1, "NUM_BRANCHES": 1, "BLOCK": "BOTTLENECK",
"NUM_BLOCKS": [1], "NUM_CHANNELS": [32], "FUSE_METHOD": "SUM"},
33             "STAGE2": {"NUM_MODULES": 1, "NUM_BRANCHES": 2, "BLOCK": "BASIC",
"NUM_BLOCKS": [2, 2], "NUM_CHANNELS": [16, 32], "FUSE_METHOD": "SUM"},
34             "STAGE3": {"NUM_MODULES": 1, "NUM_BRANCHES": 3, "BLOCK": "BASIC",
"NUM_BLOCKS": [2, 2, 2], "NUM_CHANNELS": [16, 32, 64], "FUSE_METHOD": "SUM"},
35             "STAGE4": {"NUM_MODULES": 1, "NUM_BRANCHES": 4, "BLOCK": "BASIC",
"NUM_BLOCKS": [2, 2, 2, 2], "NUM_CHANNELS": [16, 32, 64, 128], "FUSE_METHOD": "SUM"},
36             "DECONV": {"NUM_DECONVS": 0, "KERNEL_SIZE": [], "NUM_BASIC_BLOCKS": 2}},
"INIT_WEIGHTS": True}},
37         "detector": {"name": "tracknetv2", "model_path": WEIGHTS, "step": 3,
38             "postprocessor": {"name": "tracknetv2", "score_threshold": 0.5, "scales":
[0],
39                 "blob_det_method": "concomp", "use_hm_weight": True}},
40         "dataloader": {"heatmap": {"name": "binary_fixed_size", "sigmas": [2.5], "mags":
[1.0], "min_value": 0.6}},
41         "tracker": {"name": "online", "max_disp": 300},
42         "runner": {"device": "cuda", "gpus": [0]}
43     return OmegaConf.create(cfg)
44
45     def build_transforms(orig_h, orig_w):
46         from utils.image import get_affine_transform
47         c = np.array([orig_w / 2.0, orig_h / 2.0], dtype=np.float32)
48         s = max(orig_h, orig_w) * 1.0
49         trans_input = get_affine_transform(c, s, 0, [INP_W, INP_H], inv=0)
50         trans_out_inv = get_affine_transform(c, s, 0, [OUT_W, OUT_H], inv=1)
51         return trans_input.astype(np.float32), trans_out_inv.astype(np.float32)
52
53     class Detector:
54         def __init__(self, cfg, device):
55             import torch
56             from models.hrnet import HRNet
57             from detectors.postprocessor import TracknetV2Postprocessor
58             self.device = device
59             self.model = HRNet(cfg["model"]).to(device)
60             ckpt = torch.load(WEIGHTS, map_location=device, weights_only=False)
61             state = ckpt["model_state_dict"]
62             if any(k.startswith("module.") for k in state):
63                 state = {k[len("module.")]: v for k, v in state.items()}
64             missing, unexpected = self.model.load_state_dict(state, strict=False)
65             if missing or unexpected:
66                 print("[detector] load_state_dict: {} missing, {}
unexpected".format(len(missing), len(unexpected)))
67             self.model.eval()
68             self.postprocessor = TracknetV2Postprocessor(cfg)
69         def run_tensor(self, imgs, affine_mats):
70             import torch

```

```

71         with torch.no_grad():
72             imgs = imgs.to(self.device, non_blocking=True)
73             preds = self.model(imgs)
74             pp = self.postprocessor.run(preds, affine_mats)
75             results = {}
76             for bid in sorted(pp.keys()):
77                 results[bid] = {}
78                 for eid in sorted(pp[bid].keys()):
79                     dets = []
80                     for sc in sorted(pp[bid][eid].keys()):
81                         scores = pp[bid][eid][sc]["scores"]; xys = pp[bid][eid][sc]["xys"]
82                         for xy, score in zip(xys, scores):
83                             dets.append({"xy": xy, "score": score, "scale": sc})
84                     results[bid][eid] = dets
85             return results
86
87     def normalize_chw_np(chw_rgb_float):
88         out = np.empty_like(chw_rgb_float)
89         for c in range(3):
90             out[c] = (chw_rgb_float[c] - IMAGENET_MEAN[c]) / IMAGENET_STD[c]
91         return out
92
93     class CpuFrontend:
94         name = "cpu"
95         def __init__(self, video_path):
96             import cv2
97             self.cv2 = cv2
98             self.cap = cv2.VideoCapture(video_path)
99             if not self.cap.isOpened():
100                 raise RuntimeError("cannot open {}".format(video_path))
101             self.count = int(self.cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
102             self.orig_w = int(self.cap.get(cv2.CAP_PROP_FRAME_WIDTH) or 0)
103             self.orig_h = int(self.cap.get(cv2.CAP_PROP_FRAME_HEIGHT) or 0)
104             self.trans_input, self.trans_out_inv = build_transforms(self.orig_h,
self.orig_w)
105             self._pos = 0
106         def read_chw(self, index):
107             assert index == self._pos, "cpu decode forward-only ({} != {})".format(index,
self._pos)
108             ok, frame_bgr = self.cap.read()
109             if not ok:
110                 raise RuntimeError("failed to decode frame {}".format(index))
111             self._pos += 1
112             rgb = self.cv2.cvtColor(frame_bgr, self.cv2.COLOR_BGR2RGB)
113             warped = self.cv2.warpAffine(rgb, self.trans_input, (INP_W, INP_H),
flags=self.cv2.INTER_LINEAR)
114             chw = warped.transpose(2, 0, 1).astype(np.float32) / 255.0
115             return normalize_chw_np(chw)
116         def to_tensor(self, chw_np):
117             import torch
118             return torch.from_numpy(np.ascontiguousarray(chw_np))
119         def stack_window(self, chw_list):
120             import torch
121             return torch.cat([self.to_tensor(c) for c in chw_list], dim=0)
122         def release(self):
123             self.cap.release()
124
125     class GpuFrontend:
126         name = "gpu"
127         def __init__(self, video_path):
128             import torch, PyNvVideoCodec as nvc
129             self.torch = torch; self.nvc = nvc; self.gpu = 0
130             dev = torch.device("cuda", self.gpu); self.device = dev
131             self.rgb_native = True
132             try:

```

```

133         self.dec = nvc.SimpleDecoder(video_path, gpu_id=self.gpu,
output_color_type=nvc.OutputColorType.RGB)
134     except Exception as e:
135         print("[gpu] SimpleDecoder RGB failed ({!r}); fallback default".format(e))
136         self.rgb_native = False
137         self.dec = nvc.SimpleDecoder(video_path)
138     self.count = len(self.dec)
139     probe = self._decode_hwc_uint8(0)
140     self.orig_h, self.orig_w = int(probe.shape[0]), int(probe.shape[1])
141     self.trans_input, self.trans_out_inv = build_transforms(self.orig_h,
self.orig_w)
142     self.mean = torch.tensor(IMAGENET_MEAN, device=dev).view(3, 1, 1)
143     self.std = torch.tensor(IMAGENET_STD, device=dev).view(3, 1, 1)
144     # Affine sampling grid replicating WASB's cv2.warpAffine(trans_input,
INTER_LINEAR)
145     # exactly (NOT a plain resize): for each output px map back to source px via the
146     # inverse affine (trans_out_inv), normalized to [-1,1] with cv2's half-pixel
convention.
147     ys, xs = torch.meshgrid(torch.arange(INP_H, dtype=torch.float32),
148                             torch.arange(INP_W, dtype=torch.float32), indexing="ij")
149     oc = torch.stack([xs, ys, torch.ones_like(xs)], dim=-1) # [H,W,3]
output px
150     Mt = torch.tensor(self.trans_out_inv, dtype=torch.float32) # [2,3]
out->orig
151     src = oc @ Mt.transpose(0, 1) # [H,W,2] orig
px
152     gx = (2.0 * src[..., 0] + 1.0) / self.orig_w - 1.0
153     gy = (2.0 * src[..., 1] + 1.0) / self.orig_h - 1.0
154     self.grid = torch.stack([gx, gy], dim=-1).unsqueeze(0).to(dev) # [1,H,W,2]
155     def _decode_hwc_uint8(self, index):
156         torch = self.torch
157         frame = self.dec[index]
158         t = torch.from_dlpack(frame)
159         if t.dim() == 3 and t.shape[0] == 3 and t.shape[-1] != 3:
160             t = t.permute(1, 2, 0).contiguous()
161         if not self.rgb_native:
162             if t.dim() == 3 and t.shape[-1] == 3:
163                 t = t[..., [2, 1, 0]].contiguous()
164             else:
165                 raise RuntimeError("gpu fallback non-3ch frame {}; need
NV12->RGB".format(tuple(t.shape)))
166             return t
167         def read_chw(self, index):
168             import torch.nn.functional as F
169             hwc = self._decode_hwc_uint8(index)
170             chw = hwc.permute(2, 0, 1).unsqueeze(0).float() / 255.0
171             resized = F.grid_sample(chw, self.grid, mode="bilinear",
172                                     align_corners=False, padding_mode="border").squeeze(0)
# affine-warp = cv2 match
173             return (resized - self.mean) / self.std
174         def stack_window(self, chw_list):
175             return self.torch.cat(chw_list, dim=0)
176         def release(self):
177             self.dec = None
178
179     def run(frontend_cls, out_csv, cfg, device):
180         import torch
181         from trackers.online import OnlineTracker
182         fe = frontend_cls(VIDEO)
183         total = fe.count if not LIMIT else min(fe.count, LIMIT)
184         print("[{}] frames={} (processing {}), orig={}x{}".format(fe.name, fe.count, total,
fe.orig_w, fe.orig_h))
185         if total < FRAMES_IN:
186             raise RuntimeError("need >= {} frames, have {}".format(FRAMES_IN, total))
187         detector = Detector(cfg, device)

```

```
188     trans_np = fe.trans_out_inv
189     is_gpu = fe.name == "gpu"
```

13. user (2026-07-07T04:06:18.161Z)

```
60     "import torch; print('torch', torch.__version__, 'cuda', torch.cuda.is_available())"
61 )
62
63     # Decode backends wasb_infer.py understands (#506). Mirrored by its --decode-backend
choices;
64     # healthcheck rejects anything else up front so a typo can't reach argparse as rc=2 mid-
run.
65     _VALID_DECODE_BACKENDS = ("cpu", "nvdec_resident")
66
67
68     @dataclass
69     class NativeWasbConfig:
70         """Resolved settings for a Linux-native WASB run.
71
72         Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
73         over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
74         ``device``
75         defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the
perf win).
76         """
77         repo_dir: str = "~/models/WASB-SBDT" # native clone of nttcom/WASB-SBDT
78         python_bin: str = (
79             "python" # the `wasb` conda env python (invoked by path, no activate)
80         )
81         weights_path: str = "" # REQUIRED (env WASB_WEIGHTS or indexing.wasb.weights_path)
82         sport: str = "badminton"
83         device: str = "cuda" # auto | cuda | mps | cpu (auto = cuda-if-available-else-cpu)
84         scratch_dir: str = "" # frame-extraction scratch (env); "" = next to the output dir
85         timeout_sec: int = 1800 # 30 min; a hung GPU call is killed (0 = no timeout)
86         keep_frames: bool = (
87             False # retain the decoded frame cache after success (debug only)
88         )
89         # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long
clip's
90         # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a
real GPU
91         # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
here.
92         stream_video: bool = True
93         # GPU-feed tuning (indexing.wasb.batch_size / prefetch_batches). 0 = omit the flag ?
94         # wasb_infer.py's parity defaults (batch 8, no prefetch). The cloud-serving preset
opts in.
95         batch_size: int = 0
96         prefetch_batches: int = 0
97         # GPU-resident NVDEC decode (#506, indexing.wasb.decode_backend /
WASB_DECODE_BACKEND).
98         # "cpu" (default) = omit the flag ? wasb_infer.py's historical cv2 decode paths,
argv
99         # byte-identical. "nvdec_resident" = decode+resize+normalize on-GPU (torch-2.x env
only).
100         decode_backend: str = "cpu"
101
102     @classmethod
103     def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "NativeWasbConfig":
104         if isinstance(idx_cfg, dict):
105             idx_cfg = IndexingConfig(**idx_cfg)
106             w = idx_cfg.wasb
107             env = os.environ.get
108             # stream_video is tri-state in config: None/absent => native default (stream);
```

```

an
109         # explicit True/False still wins (e.g. False for a container cv2 can't seek).
110         sv = w.stream_video
111         return cls(
112             repo_dir=env("WASB_REPO_DIR") or w.repo_dir,
113             python_bin=env("WASB_PYTHON") or w.python_bin,
114             weights_path=env("WASB_WEIGHTS") or w.weights_path or "",
115             sport=w.sport,
116             device=env("WASB_DEVICE") or w.device,
117             scratch_dir=env("WASB_SCRATCH")
118             or env("RALLY_SCRATCH")
119             or env("TMPDIR")
120             or "",
121             timeout_sec=w.timeout_sec,
122             keep_frames=w.keep_frames,
123             stream_video=(True if sv is None else bool(sv)),
124             batch_size=int(w.batch_size or 0),
125             prefetch_batches=int(w.prefetch_batches or 0),
126             decode_backend=env("WASB_DECODE_BACKEND") or w.decode_backend,
127         )
128
129
130     class NativeWasbRunner(DetectorRunner):
131         """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
132         docstring."""
133         def __init__(self, cfg: NativeWasbConfig):
134             self.cfg = cfg
135             # Cached CUDA-availability probe (used to resolve device="auto"). Set by
136             healthcheck;
137             # lazily probed by run_predict if healthcheck was skipped. None = not yet
138             probed.
139             self._cuda_available: Optional[bool] = None
140
141             # --- low-level native exec (the single place tests patch)
142             -----
143             def _run(
144                 self, argv: List[str], cwd: Optional[str] = None
145             ) -> subprocess.CompletedProcess:
146                 logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
147                 return subprocess.run(
148                     argv,
149                     cwd=cwd,
150                     capture_output=True,
151                     text=True,
152                     timeout=(self.cfg.timeout_sec or None),
153                 )
154
155             # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-
156             cpu) -----
157             def _probe_cuda(self) -> bool:
158                 """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess).
159                 Any failure
160                 (missing python / timeout / import error) is treated as 'no CUDA'."""
161                 try:
162                     res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
163                 except (FileNotFoundError, subprocess.TimeoutExpired):
164                     return False
165                 return res.returncode == 0 and "cuda True" in (res.stdout or "")
166
167             def _cuda_is_available(self) -> bool:
168                 """CUDA availability, cached across healthcheck + run_predict so we probe at
169                 most once."""
170                 if self._cuda_available is None:
171                     self._cuda_available = self._probe_cuda()

```

```

166         return self._cuda_available
167
168     def _effective_device(self) -> str:
169         """The torch device to actually pass to ``wasb_infer.py``. Only ``auto`` needs
the CUDA
170         probe; an explicit cuda/mps/cpu resolves to itself (so ``device='cuda'`` is
unchanged)."""
171         if self.cfg.device == AUTO:
172             return DeviceContext(AUTO, self._cuda_is_available()).effective
173         return self.cfg.device
174
175     def _repo_src(self) -> str:
176         return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
177
178     def _copy_infer_script(self) -> bool:
179         """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
180         repo_src = self._repo_src()
181         try:
182             os.makedirs(repo_src, exist_ok=True)
183             shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
184             return True
185         except OSError as e:
186             logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
187             return False
188
189     def _rm_rf(self, path: Optional[str]) -> None:
190         """Best-effort recursive delete of a native path (post-success cleanup; never
raises)."""
191         if path:
192             shutil.rmtree(path, ignore_errors=True)
193
194     def healthcheck(self) -> Tuple[bool, str]:
195         """Verify weights + repo present and the `wasb` python imports torch (and sees a
GPU on cuda)."""
196         if not self.cfg.weights_path:
197             return False, (
198                 "WASB weights path is empty ? set WASB_WEIGHTS (or
indexing.wasb.weights_path)."
199             )
200         weights = os.path.expanduser(self.cfg.weights_path)
201         if not os.path.exists(weights):
202             return False, f"WASB weights not found at {weights}"
203         repo_src = self._repo_src()
204         if not os.path.isdir(repo_src):
205             return False, (
206                 f"WASB-SBDT repo src/ not found at {repo_src} ? clone nttcom/WASB-SBDT "
207                 f"(set WASB_REPO_DIR / indexing.wasb.repo_dir)."
208             )
209         # Fail fast on a typo'd device (the device-policy seam) ? a clear error here
beats a cryptic
210         # argparse rc=2 from wasb_infer.py at run time.
211         if self.cfg.device not in VALID_DEVICES:
212             return False, (
213                 f"unknown device {self.cfg.device!r} ? valid: {'',
'.join(VALID_DEVICES)})."
214             )
215         if self.cfg.decode_backend not in _VALID_DECODE_BACKENDS:
216             return False, (
217                 f"unknown decode_backend {self.cfg.decode_backend!r} ? valid: "
218                 f"{'', '.join(_VALID_DECODE_BACKENDS)})."
219             )
220         try:
221             res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
222         except FileNotFoundError:

```

```

223         return (
224             False,
225             f"python binary not found: {self.cfg.python_bin!r} (set WASB_PYTHON).",
226         )
227     except subprocess.TimeoutExpired:
228         return False, "native torch healthcheck timed out."
229     out = (res.stdout or "").strip()
230     if res.returncode != 0:
231         return (
232             False,
233             f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or
out).strip()}",
234         )
235     # M4: resolve the device. Cache the probe so run_predict reuses it (no second
subprocess).
236     self._cuda_available = "cuda True" in out
237     ctx = DeviceContext(self.cfg.device, self._cuda_available)
238     if ctx.cuda_required_but_missing:
239         # Unchanged pre-M4 behaviour for an EXPLICIT cuda request: hard-fail, never
a silent
240         # slow-CPU downgrade. Set indexing.wasb.device=auto for a CPU fallback.
241         return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
242     if ctx.fell_back:
243         logger.warning(
244             "device=auto: no CUDA GPU detected (%s) ? FALLING BACK TO CPU; WASB "
245             "inference will be slow. Set indexing.wasb.device=cuda to require a
GPU.",
246             out,
247         )
248     # NVDEC decode is CUDA-only: an auto?cpu fallback (or explicit cpu/mps, which
the
249     # config validator already rejects) cannot run the GPU-resident path ? fail loud
250     # instead of letting wasb_infer.py die mid-run on a paid box.
251     if self.cfg.decode_backend == "nvdec_resident" and ctx.effective != "cuda":
252         return False, (
253             f"decode_backend=nvdec_resident requires CUDA, but the effective device
is "
254             f"{ctx.effective!r} ({out}) ? NVDEC decode runs on the GPU."
255         )
256     return True, out
257
258 def run_predict(
259     self, video_win_path: str, output_win_dir: str, video_id: Optional[str] = None
260 ) -> Optional[str]:
261     """Run WASB natively on a video. Returns the path to the trajectory CSV, or
None.
262
263     Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
WSL
264     runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
the
265     same downstream contract, so the parity comparison is apples-to-apples.
266     """
267     output_dir = os.path.abspath(output_win_dir)
268     os.makedirs(output_dir, exist_ok=True)
269     # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
270     norm = str(video_win_path).replace("\\", "/")
271     stem, _ext = os.path.splitext(os.path.basename(norm))
272     # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
273     key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
274     out_csv = os.path.join(output_dir, f"{key}_wasb.csv")
275
276     if not self._copy_infer_script():
277         logger.error("Could not stage wasb_infer.py into the WASB repo src/.")

```

```

278         return None
279
280     weights = os.path.expanduser(self.cfg.weights_path)
281     argv = [
282         self.cfg.python_bin,
283         "wasb_infer.py",
284         "--video",
285         os.path.abspath(str(video_win_path)),
286     ]
287     frames_dir: Optional[str] = None
288     if self.cfg.decode_backend == "nvdec_resident":
289         # GPU-resident path (#506): wasb_infer.py decodes the video on the GPU
290         # (PyNvVideoCodec NVDEC) ? no PNG extraction and no cv2 streaming, so
291         # --stream-video nor --frames_out_dir applies and there is no frame cache to
292         # clean.
293         argv += ["--decode-backend", self.cfg.decode_backend]
294     elif self.cfg.stream_video:
295         # Fast path: decode in-process, no PNG extraction. Output is bit-identical
296         # to disk.
297         argv.append("--stream-video")
298     else:
299         scratch = self.cfg.scratch_dir or output_dir
300         frames_dir = os.path.join(scratch, f"{key}_frames")
301         argv += ["--frames_out_dir", frames_dir]

```

14. user (2026-07-07T04:06:32.441Z)

```

24: def _proc(rc=0, stdout="", stderr=""):
25:     return types.SimpleNamespace(returncode=rc, stdout=stdout, stderr=stderr)
26-
27-
28-# --- identity / no-WSL guarantee -----
29: def test_is_a_detector_runner():
30:     assert isinstance(NativeWasbRunner(NativeWasbConfig()), DetectorRunner)
31-
32-
33: def test_is_not_a_wsl_runner():
34:     """The whole point of M3.5: the native runner must carry NO WSL plumbing."""
35:     r = NativeWasbRunner(NativeWasbConfig())
36:     assert not isinstance(r, WslCommandMixin)
37:     assert not hasattr(r, "_wsl_bash")
38-
39-
40: def test_reuses_model_agnostic_windowing():
41:     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
42-
43:     assert NativeWasbRunner.parse_trajectory_csv is TrackNetRunner.parse_trajectory_csv
44:     assert (
45:         NativeWasbRunner.trajectory_to_action_windows
46:         is TrackNetRunner.trajectory_to_action_windows
47:     )
48-
49-
50-# --- config resolution (env overrides win over config over default) -----
51: def test_from_indexing_cfg_defaults():
52:     cfg = NativeWasbConfig.from_indexing_cfg({})
53:     assert cfg.device == "cuda" and cfg.python_bin == "python"
54:     # weights_path now comes from the Pydantic model default (WasbIndexConfig.weights_path)
55:     assert cfg.weights_path.endswith("wasb_badminton_best.pth.tar")
56:     assert cfg.repo_dir == "~/models/WASB-SBDT"
57:     # Native default is STREAMING (perf; bit-identical to disk, measured on a real GPU).
58:     assert cfg.stream_video is True and cfg.keep_frames is False
59:     # ?but an explicit stream_video=False is honoured (e.g. a container cv2 can't seek).

```



```

60-     assert (
61-         NativeWasbConfig.from_indexing_cfg(
62-             {"wasb": {"stream_video": False}}
63-         ).stream_video
64-         is False
65-     )
66-
111: def _ok_cfg(tmp_path, **kw):
112-     w = tmp_path / "w.pth.tar"
113-     w.write_text("weights")
114-     (tmp_path / "repo" / "src").mkdir(parents=True)
115-     return NativeWasbConfig(weights_path=str(w), repo_dir=str(tmp_path / "repo"), **kw)
116-
117-
118: def test_healthcheck_ok(tmp_path):
119-     r = NativeWasbRunner(_ok_cfg(tmp_path, device="cuda"))
120-     with patch.object(
121-         r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda True")
122-     ):
123-         ok, msg = r.healthcheck()
124-         assert ok and "torch" in msg
125-
126-
127: def test_healthcheck_empty_weights():
128-     ok, msg = NativeWasbRunner(NativeWasbConfig(weights_path="")).healthcheck()
129-     assert not ok and "weights" in msg.lower()
130-
131-
132: def test_healthcheck_weights_missing(tmp_path):
133-     (tmp_path / "repo" / "src").mkdir(parents=True)
134-     cfg = NativeWasbConfig(
135-         weights_path=str(tmp_path / "nope.pth"), repo_dir=str(tmp_path / "repo")
136-     )
137-     ok, msg = NativeWasbRunner(cfg).healthcheck()
138-     assert not ok and "not found" in msg.lower()
139-
140-
141: def test_healthcheck_repo_missing(tmp_path):
142-     w = tmp_path / "w.pth.tar"
143-     w.write_text("x")
144-     cfg = NativeWasbConfig(weights_path=str(w), repo_dir=str(tmp_path / "absent"))
145-     ok, msg = NativeWasbRunner(cfg).healthcheck()
146-     assert not ok and "repo" in msg.lower()
147-
148-
149: def test_healthcheck_torch_import_fails(tmp_path):
150-     r = NativeWasbRunner(_ok_cfg(tmp_path))
151-     with patch.object(
152-         r, "_run", return_value=_proc(1, stdout="ImportError: cannot import name 'torch' from 'torch'")
153-     ):
154-         ok, msg = r.healthcheck()
155-         assert not ok and "cannot import name 'torch' from 'torch'" in msg.lower()
156-
157-
275: def _drive_success(cfg, tmp_path, vid="abc123abc123", **patches):
276-     """Drive run_predict to success with the subprocess mocked; returns (runner, out,
277-     rm_spy, argv, cwd)."""
278-     r = NativeWasbRunner(cfg)
279-     key = f"clip__{vid[:12]}"
280-     (tmp_path / f"{key}_wasb.csv").write_text(
281-         "Frame,Visibility,X,Y\n"
282-     )
283-     # success = CSV exists
284-     with patch.object(r, "_copy_infer_script", return_value=True), patch.object(
285-         r, "_run", return_value=_proc(0)
286-     ) as run_spy, patch.object(r, "_rm_rf") as rm_spy:
287-         out = r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id=vid)
288-         argv = run_spy.call_args[0][0]
289-         cwd = run_spy.call_args.kwargs.get("cwd")
290-         return r, out, rm_spy, argv, cwd

```

```

291-def test_run_predict_builds_native_argv_no_wsl(tmp_path):
292-    # Pin disk mode here (default is now streaming) so this also covers the --frames_out_dir
path.
293-    _, out, rm, argv, cwd, key = _drive_success(
294-        NativeWasbConfig(weights_path="/m/w.pth", device="cuda", stream_video=False),
295-        tmp_path,
296-    )
297-    assert out == os.path.join(os.path.abspath(str(tmp_path)), f"{key}_wasb.csv")
298-    # No WSL / conda plumbing anywhere in the command.
299-    assert "wsl" not in argv
300-    assert not any("conda" in a or "/mnt/" in a for a in argv)
301-    # It runs the SAME inference core, device-threaded, writing the SAME CSV name as WSL.
302-    assert argv[0] == "python" and argv[1] == "wasb_infer.py"
303-    assert "--device" in argv and argv[argv.index("--device") + 1] == "cuda"
304-    assert "--weights" in argv and argv[argv.index("--weights") + 1] == "/m/w.pth"
305-    assert argv[argv.index("--out") + 1] == out
306-    # Disk mode extracts frames + cleans them on success.
307-    assert "--frames_out_dir" in argv and "--stream-video" not in argv
308-    assert cwd.endswith(os.path.join("src"))
309-    rm.assert_called_once()
310-
311-
312-def test_run_predict_default_is_stream_mode_no_frames_dir(tmp_path):
313-    # No stream_video set -> the NATIVE DEFAULT (streaming): no PNG extraction, nothing to
clean.
314-    _, out, rm, argv, _cwd, _key = _drive_success(
315-        NativeWasbConfig(weights_path="/m/w.pth"), tmp_path

```

15. user (2026-07-07T04:06:33.451Z)

```

708-    # (ModuleNotFoundError: google.cloud), on dispatch (google-cloud-run missing), or on the
first
709-    # edge-authed request (jwt missing) ? real Cloud Run failures the mock-client tests
can't see.
710-    # Lock the extras in structurally.
711-    assert ".[storage-gcs,cloud-run,auth]" in text
712-
713-
714: def test_dockerfile_uses_micromamba_not_anaconda_tos():
715:     """P3a: the WASB env is built with micromamba + conda-forge (BSD, no ToS), NOT Miniconda
+
716-     `conda tos accept` (the ?200-employee Anaconda Terms-of-Service trap, PACKAGING_PLAN.md
§3).
717-     Locked structurally so a revert to the ToS path fails CI; the env path (WASB_PYTHON) is
unchanged."""
718-     from pathlib import Path
719-
720-     text = Path("deploy/cloudrun/Dockerfile").read_text(encoding="utf-8")
721-     assert "micromamba" in text # the conda-forge env builder
722-     assert "conda-forge" in text # the BSD, ToS-free channel
723-     assert (
724-         "MAMBA_ROOT_PREFIX=/opt/conda" in text
725-     ) # keeps the env at /opt/conda/envs/wasb
726-     assert "Miniconda3" not in text # the Anaconda installer is gone
727-     assert "conda tos accept" not in text # no ToS acceptance needed (conda-forge)

```

16. user (2026-07-07T04:06:42.839Z)

```

1-"""Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING M1).
2-
3-A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
4:config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
5-knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
6-
7-Precedence (lowest ? highest), realised by the loader::

```

```

8-
9-    built-in model defaults < profile preset < config.json < env < worker --set
10-
11-This module is **pure data + pure functions** ? no IO, and (critically) **no torch import at
12-module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
--
19-from __future__ import annotations
20-
21-import copy
22-import os
23-from typing import Any, Mapping, Optional
24-
25-# Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal`` mirrors
26-# these and ``tests/test_config_deployment.py`` pins parity so the two can never drift.
27:PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28-COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
29-
30-# Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to warn
31-# when config.json explicitly overrides compute_target into a state inconsistent with the
32-profile.
33-COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
34-    "truly-local": "local_gpu",
35-    "cloud-serving": "cloud_run",
36-    "cpu-mock": "cpu_mock",
37-}
38-
39-# A preset is a partial, nested override dict over the model defaults. It touches ONLY knobs
40-# that EXIST today ? M1 is pure/additive: the new ``deployment.{profile,compute_target}``
41-plus
42-# ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones
43-extend
44-# these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
45-# Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
46-PROFILE_PRESETS: dict[str, dict[str, Any]] = {
47-    "truly-local": {
48-        "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
49-        # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU box).
50-        # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
51-        "indexing": {"skip_ai_handoff": True},
52-        "storage": {"backend": "local"},
53-    },
54-    "cloud-serving": {
55-        "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
56-        # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
57-        (D11).
58-        # wasb GPU-feed tuning: the parity defaults (batch 8, no prefetch) left the paid L4
59-        # <1% utilized on the first real serving run (2026-07-03) ? the pipeline was
60-        # CPU-feed-bound. Serving opts into the measured fast values; local/WSL unchanged.
61-        "indexing": {
62-            "detector_impl": "native",
63-            "skip_ai_handoff": False,
64-            "wasb": {"batch_size": 64, "prefetch_batches": 4},
65-        },
66-        "storage": {"backend": "gcs"},
67-    },
68-    "cpu-mock": {
69-        "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
70-        # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression profile.
71-        "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
72-        "storage": {"backend": "local"},
73-    },
74-}
75-
76-def is_known_profile(profile: str) -> bool:

```

```

74-     """True for a non-empty, recognised profile name."""
75-     return bool(profile) and profile in PROFILE_PRESETS
76-
77-
78-def preset_for(profile: str) -> dict[str, Any]:
79-     """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate the
80-     module-level table), or ``{}`` for ``""`` / an unknown profile."""
81-     return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))
82-
83-
84-def _truthy(val: Optional[str]) -> bool:
85-     """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
86-     if val is None:
87-         --
115-
116-def suggest_profile(
117-    env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
118-) -> str:
119-     """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.
120-
121-     Signals, in priority order:
122-
123-     - ``K_SERVICE`` set (Cloud Run)           ? ``"cloud-serving"``
124-     - ``CI`` truthy                           ? ``"cpu-mock"``
125-     - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
126-     - otherwise                               ? ``""`` (no strong suggestion)
127-
128-     ``probe_cuda`` defaults to ``False`` so the common config-load path stays cheap and
129-     never
130-     imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
131-     passes
132-     ``probe_cuda=True``. The result is advisory only: the caller decides whether to act on
133-     it.
134-
135-     """
136-     e = os.environ if env is None else env
137-     if e.get("K_SERVICE"):
138-         return "cloud-serving"
139-     if _truthy(e.get("CI")):
140-         return "cpu-mock"
141-     if probe_cuda and _cuda_available():
142-         return "truly-local"
143-     return ""

```

17. user (2026-07-07T04:07:58.444Z)

```

14:VIDEO = osp.expanduser(osp.environ.get("T2_VIDEO", osp.join(T2, "g1.mp4")))
18:FRAMES_IN = 3; FRAMES_OUT = 3
19:INP_W, INP_H = 512, 288; OUT_W, OUT_H = 512, 288; SCALE = 0
22:LIMIT = int(osp.environ.get("T2_LIMIT", "0") or "0")
27:     "model": {"name": "hrnet", "frames_in": FRAMES_IN, "frames_out": FRAMES_OUT,
28:     "inp_height": INP_H, "inp_width": INP_W, "out_height": OUT_H, "out_width": OUT_W,
49:     trans_input = get_affine_transform(c, s, 0, [INP_W, INP_H], inv=0)
50:     trans_out_inv = get_affine_transform(c, s, 0, [OUT_W, OUT_H], inv=1)
113:     warped = self.cv2.warpAffine(rgb, self.trans_input, (INP_W, INP_H),
114:     flags=self.cv2.INTER_LINEAR)
147:     ys, xs = torch.meshgrid(torch.arange(INP_H, dtype=torch.float32),
148:     torch.arange(INP_W, dtype=torch.float32), indexing="ij")
183:     total = fe.count if not LIMIT else min(fe.count, LIMIT)
185:     if total < FRAMES_IN:
186:         raise RuntimeError("need >= {} frames, have {}".format(FRAMES_IN, total))
197:     if len(window) < FRAMES_IN:
199:     if len(window) > FRAMES_IN:
211:         print("{} windows={}/{} | {:.1f} win/s | {:.0f}s".format(fe.name, n_windows,
212:         total - FRAMES_IN + 1, fps, el))

```

18. user (2026-07-07T04:07:59.387Z)

```
9:from __future__ import annotations
11:from typing import Any, List, Literal, Optional
13:from pydantic import BaseModel, Field, field_validator, model_validator
15:Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
91:     device: str = "cuda" # auto | cuda | mps | cpu
131:     decode_backend: Literal["cpu", "nvdec_resident"] = "cpu"
133:     @model_validator(mode="after")
180:     substrate: Literal["wasb", "tracknet"] = "wasb"
198:     net_axis: Literal["x", "y"] = "y"
311:     @model_validator(mode="after")
349:     position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = (
421:     backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
432:     input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
468:     profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
471:     compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
```

19. assistant (2026-07-07T04:08:08.208Z)

All static checks so far pan out. Now let me actually run the new/changed tests to ground the validity claims.

20. user (2026-07-07T04:08:17.284Z)

```
.....S..... [100%]
65 passed, 1 skipped in 4.91s
```

21. user (2026-07-07T04:08:25.275Z)

```
.. [100%]
2 passed, 19 deselected in 0.27s
```

22. assistant (2026-07-07T04:09:48.672Z)

Let me check the import chain the parity test needs inside the wasb env (its documented invocation environment), and the tail of `run\_predict` for the rm assertion validity.

23. user (2026-07-07T04:09:51.208Z)

```
==> backend/pipeline/detectors/__init__.py <==
"""Low-level moving-object detectors that run outside the main Python process.

Houses:
- DetectorRunner ABC (the seam for WSL vs future Linux-native trajectory detectors).
- TrackNetRunner / WasbRunner (current WSL adapters; both inherit DetectorRunner).
---
37:import logging
38:import os
39:import shutil
40:import subprocess
41:from dataclasses import dataclass
42:from typing import List, Optional, Tuple
44:from backend.config.models import IndexingConfig
45:from backend.pipeline.detectors.base import DetectorRunner
46:from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext
49:from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
```

24. user (2026-07-07T04:09:52.143Z)

```
        argv += ["--frames_out_dir", frames_dir]
        # M4: resolve device="auto" to the effective device (cuda-if-available-else-cpu); an
        # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to wasb_infer.py.
        device = self._effective_device()
```

```

if self.cfg.decode_backend == "nvdec_resident" and device != "cuda":
    # Mirrors the healthcheck gate for callers that skipped it (e.g. auto?cpu on a
    # GPU-less box): NVDEC decode cannot run without CUDA ? fail before the subprocess.
    logger.error(
        "decode_backend=nvdec_resident requires CUDA, but the effective device is "
        "%r ? aborting before the WASB subprocess.",
        device,
    )
    return None
argv += [
    "--weights",
    weights,
    "--sport",
    self.cfg.sport,
    "--device",
    device,
    "--out",
    out_csv,
]
# GPU-feed tuning: pass only explicit non-zero values so the default argv (and its
# byte-parity guarantee) is unchanged when the knobs are unset.
if self.cfg.batch_size:
    argv += ["--batch-size", str(self.cfg.batch_size)]
if self.cfg.prefetch_batches:
    argv += ["--prefetch-batches", str(self.cfg.prefetch_batches)]

logger.info(
    "Running native WASB inference (device=%s%s) on %s ...",
    device,
    " [auto?fallback]" if self.cfg.device == AUTO and device != "cuda" else "",
    os.path.basename(norm),
)
try:
    res = self._run(argv, cwd=self._repo_src())
except FileNotFoundError:
    logger.error(
        "python binary not found: %r (set WASB_PYTHON).", self.cfg.python_bin
    )
    return None
except subprocess.TimeoutExpired:
    logger.error(
        "WASB native inference timed out after %ss.", self.cfg.timeout_sec
    )
    return None
if res.returncode != 0:
    logger.error(
        "WASB native inference failed:\n%s",
        (res.stderr or res.stdout or "")[-2000:],
    )
    return None
if not os.path.exists(out_csv):
    logger.error(
        "WASB native finished but expected CSV not found at %s", out_csv
    )
    return None
logger.info("WASB trajectory CSV: %s", out_csv)

```

Success ? the CSV is the durable output; the decoded frame cache is a pure intermediate.

25. user (2026-07-07T04:10:08.765Z)

```

17:from .base import DetectorRunner
19:# Re-export the ABC so "from backend.pipeline.detectors import DetectorRunner" works.
22:# The concrete runners are imported directly where needed (hybrids, eval/*).

```

```
23:# This keeps heavy optional deps from being pulled at import time of the package.
---BACKEND-INIT---
cat: backend/__init__.py: No such file or directory
---PIPE-INIT---
cat: backend/pipeline/__init__.py: No such file or directory
---CONFTEST---
```

26. user (2026-07-07T04:10:48.773Z)

```
55:_INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
183:         shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
---
backend/config/__init__.py
9:import os
11:from .loader import (
17:from .models import ( # re-export common ones
26:from .presets import PROFILE_PRESETS, probe_cuda_available, suggest_profile
27:from .startup import (
---BASE---
backend/pipeline/detectors/base.py:69:import logging
backend/pipeline/detectors/base.py:70:import os
backend/pipeline/detectors/base.py:71:import shlex
backend/pipeline/detectors/base.py:72:import subprocess
backend/pipeline/detectors/base.py:73:from abc import ABC, abstractmethod
backend/pipeline/detectors/base.py:74:from typing import Any, Callable, List, Optional, Tuple
backend/pipeline/detectors/tracknet_runner.py:17:import csv
backend/pipeline/detectors/tracknet_runner.py:18:import logging
backend/pipeline/detectors/tracknet_runner.py:19:import os
backend/pipeline/detectors/tracknet_runner.py:20:import shlex
backend/pipeline/detectors/tracknet_runner.py:21:import subprocess
backend/pipeline/detectors/tracknet_runner.py:22:from dataclasses import dataclass
backend/pipeline/detectors/tracknet_runner.py:23:from typing import Any, Dict, List, Optional,
Tuple
backend/pipeline/detectors/tracknet_runner.py:25:from backend.config.models import
IndexingConfig
backend/pipeline/detectors/tracknet_runner.py:26:from backend.pipeline.detectors.base import (
```

27. assistant (2026-07-07T04:13:01.191Z)

Let me verify the exact import chain for the parity-test finding (what the wasb env would need), and double-check `backend.config` submodule imports.

28. user (2026-07-07T04:13:07.963Z)

```
backend/config/loader.py:11:import json
backend/config/loader.py:12:import logging
backend/config/loader.py:13:import os
backend/config/loader.py:14:from pathlib import Path
backend/config/loader.py:17:from pydantic import BaseModel
backend/config/loader.py:21:from .models import AppConfig
backend/config/loader.py:22:from .presets import (
backend/config/startup.py:36:import logging
backend/config/startup.py:37:import os
backend/config/startup.py:38:from dataclasses import dataclass
---
wasb_infer import: OK (stdlib-only)
native_wasb_runner import: OK
```

29. user (2026-07-07T04:13:20.586Z)

```
wasb_infer import: OK (stdlib-only)
native_wasb_runner import FAILED: ModuleNotFoundError No module named 'pydantic' (simulated wasb
env)
```

30. assistant (2026-07-07T04:13:59.909Z)

All checks grounded. Let me get exact line numbers in the deploy files for the findings.

31. user (2026-07-07T04:14:01.699Z)

```
11:# PyNvVideoCodec for the GPU-resident decode backend (indexing.wasb.decode_backend=
13:# PyNvVideoCodec needs (compute-only driver images ship neither).
35:echo "[gpu] [2/6] NVDEC/NVENC driver userspace libs (#506: PyNvVideoCodec needs BOTH)"
44: sh /tmp/drv.run --extract-only --target /tmp/drv-extract >/dev/null
66:echo "[gpu] [4/6] conda env 'wasb' (py3.10 + torch 2.6.0+cu124 + PyNvVideoCodec ? #506
stack)"
71:python -m pip install -q PyNvVideoCodec
75:python -m pip install -q hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm
pyyaml matplotlib numpy || true
93:     import PyNvVideoCodec as pncv
94:     print("[gpu]         PyNvVideoCodec", getattr(pncv, "__version__", "?"), "OK")
96:     print("[gpu]         !! PyNvVideoCodec import failed:", repr(e)[:150])
---DOCKERFILE---
11:# * a conda `wasb` env (py3.10 + torch 2.6.0+cu124 + PyNvVideoCodec ? the #506 modernized
stack,
21:# the video-codec libs ? no libnvcuvid.so.1, no libnvidia-encode.so.1 ? and PyNvVideoCodec
needs
53:# PyNvVideoCodec dlopens. Harmless when decode_backend=cpu; required for nvdec_resident.
69:# PyNvVideoCodec (MIT ? the practical NVDEC path; TorchCodec needs a matching ffmpeg build
and could
81:    && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q PyNvVideoCodec \
```

32. user (2026-07-07T04:14:53.380Z)

Structured output provided successfully